

Relatório de Auditoria de Segurança — Rufino Project

Plataforma de gestão de pessoas, documentos e pagamento de boletos — monorepo (3 APIs .NET, 2 apps Flutter, infraestrutura Azure/Docker)

Escopo auditado

- server/Services/BillPayment — API .NET 10 (76 actions, 10 repositórios, 27 queries, 6 workers)
- server/Services/PeopleManagement — API .NET 8 (17 controllers, MediatR, Hangfire, Puppeteer, ZapSign, Evolution API)
- server/Services/TenantManagement — API .NET (emissor do TenantId, provisionamento no Keycloak)
- client/rufino_v2 e client/rufino — apps Flutter (web/mobile/desktop) e nginx do build web
- Deploy e configuração: docker-compose (raiz e por BC), Dockerfiles, .github/workflows, codemagic.yaml, azure/container-apps (Bicep + deploy.ps1), appsettings*.json, utils/KeyCloakConfig (realm), keycloak-themes
- Histórico git completo (1.344 commits) para segredos já removidos da árvore

DATA

28 de agosto de 2026

MÉTODO

Revisão manual de código-fonte (white-box), sem execução contra ambiente; 5 categorias adaptadas à stack; só achados verificados em arquivo:linha

REVISÃO AUDITADA

branch feature/bill-payment, commit 2ed33448 (worktree rufino-bill-payment)

ACHADOS

32 (2 crítica, 3 alta, 9 média, 12 baixa, 6 informativa)

Nota metodológica

O projeto não usa Supabase nem RLS: cada API isola tenants em código. O mecanismo é a policy montada por [ProtectedResource(recurso, escopo)], que soma dois requisitos — RouteAccessRequirement (o parâmetro de rota {tenantId} / {company} precisa constar no claim `tenants`/`bp\_tenants`/`companies` do JWT do Keycloak) e ProtectedResourceRequirement (consulta UMA ao Keycloak pelo escopo). Como a rota sem o parâmetro passa em silêncio e o guard só confere a rota (não o objeto), a segunda camada obrigatória é o filtro explícito por TenantId/CompanyId em cada handler, repositório e query — não existe HasQueryFilter global em nenhum DbContext. As cinco categorias pedidas foram mapeadas assim:

CATEGORIA PEDIDA	COMO FOI MAPEADA PARA A STACK DETECTADA
1. Banco sem tranca — isolamento de Tenant/Company	Seto de Tenant/Company (a) toda rota protegida declara o parâmetro exato ({tenantId}/{company}); (b) toda listagem, agregação, dashboard, exportação, batch e download filtra por TenantId/CompanyId da rota; (c) subqueries e joins que seguem IDs relacionados; (d) mecanismos transversais sem tenant (ledger de idempotência, provisionamento de claims no Keycloak).
2. Permissão definida no navegador	Flutter não lê papéis do JWT: pede um RPT UMA ao Keycloak e esconde botões por hasPermission(recurso, escopo). Cruzamos os 30 gates de UI (bill_payment, tenant_management, people-management) com o atributo [ProtectedResource] do endpoint chamado, e listamos superfícies do servidor fora do Keycloak (Hangfire, Swagger, TestController, CORS, FallbackPolicy).
3. IDOR — referência direta a objetos sem TOC	Objetos sem TOC: gerem de posse e rota das 3 APIs (76 no BillPayment, ~110 no PeopleManagement, 17 no TenantManagement). Para cada ID recebido em path/query/body, seguimos handler → repositório/query e confirmamos se o lookup é `TenantId && Id`. IDs de objetos relacionados vindos do corpo foram checados quanto à validação de pertencimento.
4. Chaves expostas (hardcode)	Grep por credenciais na árvore (appsettings, compose, Bicep, template.env, CI, scripts, realm export, Flutter/secrets, bundle web), defaults `\${VAR:-x}`, validação de startup (ValidateOnStart/IValidateOptions) e histórico git (git log -S / git show em 1.344 commits). Valores reais aparecem mascarados neste relatório.
5. Inputs sem tratamento (XSS/HTML injection)	innerHTML: checamos flutter_html/HtmlWidget, WebView, dart:html, url_launcher com URL de API, index.html. No backend, o equivalente é a geração de PDF por Chromium headless (PuppeteerSharp) a partir de templates HTML enviados pelo tenant e de valores de funcionário interpolados sem encoding, além de e-mails/WhatsApp e temas Keycloak (.ftl).

Stack detectada

Linguagem / runtime	C# 12–13 em .NET 8 (PeopleManagement) e .NET 10 (BillPayment, TenantManagement); Dart/Flutter 3 (clientes)
Framework web	ASP.NET Core MVC (controllers + [ApiController]); MediatR no PeopleManagement, mediator próprio nos outros dois
ORM / acesso a dados	EF Core + Npgsql (PostgreSQL 16), um schema por BC; sem SQL cru na aplicação (exceto filas de worker parametrizadas)

Autenticação / autorização	Keycloak (JWT Bearer com issuer/audience/lifetime) + UMA ticket (RPT) por [ProtectedResource]; guard de rota por claim tenants/bp_tenants/companies
Frontend	Flutter (rufino_v2: pacotes rufino_core, bill_payment, tenant_management; web servido por nginx) e app legado rufino
Integrações	ZapSign (assinatura + webhook), Evolution API (WhatsApp), Microsoft Graph (caixa de e-mail), Asaas, Gemini, S3/MinIO/Garage, Azure Blob, Hangfire (jobs)
Deploy	Docker Compose (dev), Azure Container Apps via GitHub Actions (PeopleManagement, Evolution API por Bicep/deploy.ps1), Dokploy (BillPayment/TenantManagement, por ADR-009), Codemagic (iOS)

Regras de classificação

- Só entrou no relatório o que foi lido no código desta revisão; o comportamento de bibliotecas de terceiros não lidas (ex.: Hangfire.Dashboard.BasicAuthorization com senha vazia) é marcado como não verificado.
- Severidade combina impacto e condição de exploração: crítica = execução/leitura de infraestrutura ou segredo real exposto; alta = leitura/escrita cross-tenant com privilégio comum; média = exige privilégio elevado, configuração adicional ou tem impacto contido; baixa = hardening/defesa em profundidade; informativa = sem exploração hoje, mas vale registrar.
- Pontos fortes só entram com evidência arquivo:linha — eles provam a cobertura da auditoria tanto quanto os achados.
- Linhas referem-se ao commit 2ed33448; trechos são cópias literais (segredos mascarados).

Sumário

1. Resumo executivo
2. Pontos fortes e pontos fracos
3. Achados detalhados por categoria
4. Recomendações priorizadas
5. Issues para o GitHub

1. Resumo executivo

A auditoria cobriu as três APIs, os dois apps Flutter, os arquivos de deploy e o histórico git. O desenho de autorização é sólido e consistente — policy dupla (guard de rota + UMA), TenantId sempre da rota, filtro explícito por tenant em todos os repositórios do BillPayment e nas listagens do PeopleManagement, segredos fora do appsettings atual. Os riscos centrais estão em duas frentes do PeopleManagement: (1) a geração de PDF executa HTML/JavaScript enviado pelo tenant — e valores de funcionário sem encoding — dentro de um Chromium sem sandbox com origem file://, o que permite ler arquivos do container e alcançar a rede interna; (2) duas rotas fogem do guard de tenant (CompanyController e o webhook da ZapSign), permitindo ler/editar qualquer empresa e forjar assinaturas com download de URL arbitrária. Na frente de segredos, quatro credenciais reais (dois client secrets do Keycloak, um token ZapSign e o secret do app) estão recuperáveis no histórico do repositório hospedado no GitHub. O restante são defaults de credencial em compose/appsettings, ausência de validação de startup, Hangfire fora do Keycloak e itens de hardening.



Leitura rápida

- 2 achados críticos: HTML do tenant executado em Chromium no servidor (XSS-01) e segredos reais no histórico git (SEC-01).
- 3 achados altos: CompanyController sem guard de tenant (IDR-01), webhook ZapSign sem escopo/assinatura com SSRF (IDR-02), valores de funcionário injetados no HTML sem encoding (XSS-02).
- BillPayment: nenhum IDOR ou vazamento cross-tenant; único ponto é a importação JSON aceitar StorageKey/Sourcelid do corpo (IDR-03, média).
- TenantManagement: convite pode ser sequestrado por quem troca o próprio e-mail no Keycloak (ISO-06, média).
- Os 30 gates de permissão do Flutter têm equivalente exato no servidor — a categoria 2 não produziu falha de privilégio, só superfícies fora do Keycloak.

- Frontend Flutter sem innerHTML/eval; temas Keycloak sem .ftl; notificações em texto puro — XSS clássico não se aplica.

2. Pontos fortes e pontos fracos

2.1 Pontos fortes (verificados com evidência)

Policy dupla em todo endpoint protegido (guard de rota + UMA)

Cada [ProtectedResource] gera policy com RequireAuthenticatedUser + RouteAccessRequirement + ProtectedResourceRequirement nos três BCs; token rejeitado vira 401 e Keycloak fora vira 503, nunca 403 ambíguo.

Evidência: BillPayment.API/Authorization/AuthorizationExtesion.cs:33-41; PeopleManagement.API/Authorization/AuthorizationExtesion.cs:21-30; TenantManagement.API/Authorization/AuthorizationExtesion.cs:32-36; */AuthorizationResultHandler.cs:36-61

BillPayment: 76/76 actions sob api/v1/{tenantId:guid} com [ProtectedResource]; zero [Authorize] cru

Único anônimo é o health ([AllowAnonymous] explícito). Teste de erosão varre o EndpointDataSource e falha se uma rota api/v1 nascer sem {tenantId} ou sem [ProtectedResource].

Evidência: BillPayment.API/Controllers/*.cs; HealthController.cs:12; BillPayment.IntegrationTests/Authorization/EndpointProtectionTests.cs:34-62; RouteGuardTests.cs

BillPayment: toda busca por Id é `TenantId && Id`, em 10 repositórios e 27 queries

Listagens filtram TenantId antes do cursor; as três travessias cross-tenant documentadas devolvem só bool; workers obtêm TenantId da própria linha reivindicada; download de artefato tem dupla barreira (banco + prefixo tenants/{tenantId}/ no S3).

Evidência: BillPayment.Infra/Repositories/*.cs (ex.: BillRepository.cs:25-27, PayeeRepository.cs:17-19); BillPayment.Application/Queries/Bills/BillQueries.cs:26,70,159; Infra/Storage/S3AttachmentStorage.cs:135,145-149

TenantId/CompanyId nunca vem do corpo da requisição

Todos os Models expõem ToCommand(Guid tenantId/company) recebendo o valor da rota; nenhum Model declara TenantId. No TenantManagement o claim gravado no Keycloak deriva do {id} da rota.

Evidência: BillPayment.Application/Models//*.cs; PeopleManagement (ex.: CreateEmployeeCommand.cs:17, AddListenEventCommand.cs:9); TenantManagement.Application/Models/Tenants/TenantModels.cs:46-49; KeycloakTenantAccessProvisioner.cs:64,144

PeopleManagement: handlers de edição e fluxos de documento exigem Id && CompanyId (ou a tripla DocumentId/EmployeeId/CompanyId)

Todos os Alter*Employee, Edit*, MarkAs*DocumentUnit e serviços de documento/assinatura filtram pela company da rota; blobs têm nome gerado no servidor e container = CompanyId.

Evidência: AlterAddressEmployeeCommandHandler.cs:22 ... MarkEmployeeAsInactive:21; DocumentService.cs:54,277,343,376,430,498; SignDocumentService.cs:44,83; DocumentQueries.cs:120,154; BatchDownloadQueries.cs:218

Os 30 gates de permissão do Flutter têm o escopo equivalente no servidor

O app não decodifica papéis: consulta o RPT UMA por audiência e trata o gate como conveniência ('The server decides anyway'). O seletor de tenant usa /me/tenants, resolvido pelo e-mail do token, sem parâmetro.

Evidência: client/rufino_v2/packages/rufino_core/lib/src/auth/permission_api_service.dart:55-66; packages/bill_payment/lib/src/ui/bill_payment_routes.dart:479-493; TenantManagement.API/Controllers/MeController.cs:27-32

JWT validado com issuer, audience e lifetime; identidade do aprovador só do `sub`

O fallback por header x-user-id do BillPayment foi removido; a chamada UMA propaga o token do próprio chamador.

Evidência: */Authentication/KeycloakExtesion.cs:18-28; BillPayment.API/Controllers/BaseController.cs:141-142; AccessTokenPropagationHandler.cs:26-33

Segredos fora da árvore atual e fora do container

.env ignorado e nunca versionado; .dockerignore exclui **/.env e secrets/; CI usa só \${{ secrets.* }} e secretfef; BillPayment exige a master key com \${VAR:?} e falha alto quando cofre/storage não estão configurados; TenantManagement marca provisionamento como Failed sem ClientSecret.

Evidência: .gitignore:28-32; server/.dockerignore:3; deploy-people-management-api.yml:48; server/Services/BillPayment/docker-compose.yml:25; BillPayment.Infra/InfraDependencies.cs:200,423; TenantManagement.Infra/InfraDependencies.cs:65

Frontend sem vetores de XSS clássico

Nenhum dart:html/innerHTML/eval/WebView; HtmlWidget do e-mail não executa script e bloqueia imagens remotas até o usuário optar; url_launcher só abre http/https; temas Keycloak são só CSS/propriedades (sem .ftl); notificações WhatsApp/Graph em texto puro.

Evidência: packages/bill_payment/lib/src/ui/shared/email_viewer_screen.dart:203-224; client/rufino_v2/lib/app.dart:1078-1089; keycloak-themes/rufino/**; GraphNotificationSender.cs:101; WhatsAppService.cs:33-37

Sem SQL cru na aplicação; extração de zip resistente a zip-slip

Filtros de busca parametrizados pelo EF; ZipArchive.ExtractToDirectory do .NET rejeita entradas fora do destino.

Evidência: PeopleManagement.Application/Queries/Employee/EmployeeQueries.cs:40,45; TenantQueries.cs:151-157; PeopleManagement.Infra/Services/LocalStorageService.cs:19-20

2.2 Pontos fracos (riscos centrais)

Geração de PDF executa HTML/JS controlado pelo tenant em Chromium sem isolamento

Template enviado por upload e valores de funcionário sem encoding vão para page.SetContentAsync num Chromium --no-sandbox com origem file:// e JavaScript ligado. Leitura de appsettings.json (que traz connection string), /proc/self/environ e SSRF para a rede interna a partir de um privilégio de tenant.

Achados: XSS-01, XSS-02, XSS-05

Duas rotas do PeopleManagement fogem do guard de tenant

CompanyController (api/v1/company) e o webhook ZapSign (api/v1/Document/webhook) não têm {company} na rota; o guard passa em silêncio e os handlers buscam por Id global. Resultado: ler/editar qualquer empresa e forjar 'documento assinado' com download de URL arbitrária.

Achados: IDR-01, IDR-02

Segredos reais recuperáveis no histórico git

Client secrets do Keycloak (people-management-api, zapsign, rufino-app) e token da API ZapSign foram commitados entre 2024-08 e 2025-07 e continuam acessíveis por git show num repositório hospedado no GitHub.

Achados: SEC-01

Isolamento do PeopleManagement depende de cada query — e algumas seguem IDs sem filtro

IDs relacionados (RoleId, WorkPlaceId, DepartmentId, PositionId, DocumentGroupId) são aceitos do corpo sem validar pertencimento, e joins/subqueries os seguem sem CompanyId, levando salário/CBO de outra empresa para dentro de documentos. O ledger de idempotência é global e o header ausente vira Guid.Empty, anulando comandos entre tenants.

Achados: ISO-01, ISO-02, ISO-03, ISO-04, ISO-05

Defaults de credencial e ausência de validação de startup

Senha do Postgres no appsettings base das 3 APIs e em todos os composes, admin/Admin no Keycloak local, MinIO fraco, template.env da Evolution API com senha-aqui e CORS *, Hangfire com senha vazia por padrão — nenhuma API rejeita segredo vazio ao subir.

Achados: SEC-02, SEC-03, SEC-04, SEC-06, PRM-01

3. Achados detalhados por categoria

Tabela-síntese seguida da ficha de cada achado (trecho literal, por que é explorável, condições de explorabilidade). Linhas referem-se à revisão auditada.

3.1 Banco sem tranca — isolamento de tenant/dono

Mecanismo: RouteAccessRequirement + filtro manual por tenant. BillPayment está íntegro (76/76 actions, todos os repositórios e queries filtram TenantId). PeopleManagement filtra todas as listagens principais, mas segue IDs relacionados sem filtro e compartilha o ledger de idempotência entre tenants.

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Média	server/Services/PeopleManagement/PeopleManagement.Infra/Repository/RoleRepository.cs:13-16 (+ AlterRoleEmployeeCommandHandler.cs:24-27, DepartmentQueries.cs:25-34, DocumentGroupQueries.cs:85-88, RequireDocumentsQueries.cs:106-108, DocumentTemplateQueries.cs:95-96, EmployeeQueries.cs:26, BatchDownloadQueries.cs:20-26)	ISO-01 — PeopleManagement: IDs relacionados aceitos do corpo sem validar tenant, e joins que os seguem sem CompanyId
Média	server/Services/PeopleManagement/PeopleManagement.Application/Commands/Identified/IdentifiedCommandHandler.cs:19-26 (+ EmployeeController.cs:42, RequestManager.cs:11-17, ClientRequestMap.cs:12-14)	ISO-03 — PeopleManagement: ledger de idempotência global sem tenant, e x-requestid ausente vira Guid.Empty
Média	server/Services/TenantManagement/TenantManagement.Infra/Identity/KeycloakTenantAccessProvisioner.cs:66-82 (+ 184-195, 135-153; utils/KeycloakConfig/RufinoRealm/realm-export-2026-08-18.json:35,3109)	ISO-06 — TenantManagement: claim `tenants` é gravado em qualquer conta que tenha o e-mail — convite sequestrável por troca de e-mail self-service
Baixa	server/Services/PeopleManagement/PeopleManagement.Application/Queries/RequireDocuments/RequireDocumentsQueries.cs:27-30	ISO-02 — PeopleManagement: GetAllWithDocumentList busca o funcionário sem CompanyId
Informativa	server/Services/BillPayment/BillPayment.Infra/Idempotency/RequestManager.cs:15-16 (+ TenantManagement.Infra/Persistence/ClientRequest.cs:92, RequestManager.cs:77-84)	ISO-04 — BillPayment e TenantManagement: ledger de idempotência também é global (chave escolhida pelo cliente)
Informativa	server/Services/BillPayment/BillPayment.Infra/Persistence/BillPaymentDbContext.cs:51-76 (grep HasQueryFilter vazio nos 3 BCs)	ISO-05 — Nenhum DbContext tem HasQueryFilter global: o isolamento depende de cada query

ISO-01 Média

PeopleManagement: IDs relacionados aceitos do corpo sem validar tenant, e joins que os seguem sem CompanyId

server/Services/PeopleManagement/PeopleManagement.Infra/Repository/RoleRepository.cs:13-16 (+ AlterRoleEmployeeCommandHandler.cs:24-27, DepartmentQueries.cs:25-34, DocumentGroupQueries.cs:85-88, RequireDocumentsQueries.cs:106-108, DocumentTemplateQueries.cs:95-96, EmployeeQueries.cs:26, BatchDownloadQueries.cs:20-26)

```
// RoleRepository.cs:13-16 – filtra o EMPREGADO pela company, não a ROLE
var query = from e in _context.Employees
            join r in _context.Roles on e.RoleId equals r.Id
            where e.Id == employeeId && e.CompanyId == companyId
            select r;

// AlterRoleEmployeeCommandHandler.cs:24-27 – RoleId do corpo vai direto para o agregado
var employee = await _employeeRepository.FirstOrDefaultAsync(x => x.Id == request.EmployeeId &&
x.CompanyId == request.CompanyId, ...)
    ?? throw new DomainException(...);
employee.RoleId = request.RoleId;
```

Por que é explorável: Um usuário com employee:edit na empresa A informa o RoleId (GUID) de uma Role da empresa B em PUT /api/v1/{A}/employee/Role. Nada valida que a Role pertence a A. Em seguida, GET employee/list, batch-download/employees e principalmente a geração de documentos (RecoverRoleInfoToDocumentTemplateService.cs:17-29) seguem o join sem filtrar r.CompanyId e trazem nome, CBO e remuneração (salário base) da Role alheia para o JSON/PDF do documento. O mesmo padrão vale para WorkPlaceld, Position.DepartmentId, Role.PositionId, DocumentTemplate.DocumentGroupId e RequireDocuments.DocumentsTemplatesIds (subqueries em DepartmentQueries.cs:25-34, DocumentGroupQueries.cs:87, RequireDocumentsQueries.cs:106-108 sem CompanyId).

Condições de explorabilidade: Exige conhecer GUIDs de objetos de outra empresa (ex.: usuário que perdeu acesso a uma empresa e guardou IDs; GUIDs circulam em URLs e logs). Privilégio: employee:edit / role:create / position:create / document-template:create.

ISO-03 Média

PeopleManagement: ledger de idempotência global sem tenant, e x-requestid ausente vira Guid.Empty

server/Services/PeopleManagement/PeopleManagement.Application/Commands/Identified/IdentifiedCommandHandler.cs:19-26 (+ EmployeeController.cs:42, RequestManager.cs:11-17, ClientRequestMap.cs:12-14)

```
// EmployeeController.cs:42 – header opcional; Guid é value type → ausente = Guid.Empty
public async Task<ActionResult<CreateEmployeeResponse>> Create([FromRoute] Guid company,
[FromBody] CreateEmployeeModel request, [FromHeader(Name = "x-requestid")] Guid requestId)

// IdentifiedCommandHandler.cs:19-26
var alreadyExists = await _requestManager.ExistAsync(request.Id);
if (alreadyExists)
{
    _logger.LogInformation("Request {Id} already exists", request.Id);
    return CreateResultForDuplicateRequest(); // default! – não executa e não erra
}
await _requestManager.CreateRequestForCommandAsync<T>(request.Id);
```

Por que é explorável: A tabela client_requests tem só Id/Name/Time, sem CompanyId nem usuário. (1) A primeira requisição sem header, de qualquer tenant, grava Guid.Empty; a partir daí TODA requisição sem header de TODOS os tenants (inclusive o webhook ZapSign, que só envia Authorization) retorna default sem executar — negação de serviço silenciosa entre tenants. (2) Quem conhecer/adivinhar o x-requestid de outro tenant pode registrá-lo antes e anular o comando dele. BaseController.ValidityRequestId (BaseController.cs:51-55) existe, nunca é chamado e está com a condição invertida; só CompanyController valida o header.

Condições de explorabilidade: Qualquer usuário autenticado de qualquer tenant; para o item (2) é preciso conhecer o GUID alheio.

ISO-06 Média

TenantManagement: claim `tenants` é gravado em qualquer conta que tenha o e-mail — convite sequestrável por troca de e-mail self-service

server/Services/TenantManagement/TenantManagement.Infra/Identity/KeycloakTenantAccessProvisioner.cs:66-82 (+ 184-195, 135-153; utils/KeycloakConfig/RufinoRealm/realm-export-2026-08-18.json:35,3109)

```
var user = await FindUserByEmailAsync(email, cancellationToken); // users?email=...&exact=true
→ users[0]
var created = false;
if (user is null)
{
    await CreateUserAsync(email, tenant, products, cancellationToken);
    ...
}
else
{
    await WriteAccessAsync(user, tenant, products, cancellationToken); // grava
    tenants/bp_tenants/pm_tenants sem olhar EmailVerified
}
```

Por que é explorável: O realm exportado tem `verifyEmail=false` (L35), `duplicateEmailsAllowed=false` e o User Profile permite ao próprio usuário editar `email` (L3109: `permissions.edit = [admin, user]`) com `account-console` habilitado. Qualquer conta existente no realm troca o próprio e-mail para `fulano@empresa-b.com` ANTES do convite. Quando o operador chama `POST /tenants/{B}/members {email}`, o adapter encontra a conta do atacante e grava `tenants={B}` nela; nenhum convite é enviado (só quando `created`) e a vítima nunca poderá ser criada com aquele e-mail. `GET /me/tenants` (MeController.cs:27-32) também é chaveado pelo `claim email` do token.

Condições de explorabilidade: Conta prévia no realm rufino; conhecer o e-mail alvo antes do convite; Keycloak com a configuração exportada. O código não exige `emailVerified` nem compara `IdentityUserId`.

ISO-02 Baixa

PeopleManagement: GetAllWithDocumentList busca o funcionário sem CompanyId

server/Services/PeopleManagement/PeopleManagement.Application/Queries/RequireDocuments/RequireDocumentsQueries.cs:27-30

```
var employeeQuery =
    from em in context.Employees.AsNoTracking()
    where em.Id == employeeId
    select em.GetAllPossibleAssociationIds();
```

Por que é explorável: `GET /api/v1/{company}/requireddocuments/withdocuments/{employeeId}` resolve as associações (cargo, função, local) de um funcionário de QUALQUER empresa. As linhas 35 e 44 limitam o retorno aos `RequireDocuments/Documents` da company da rota, então o que vaza é apenas 'quais dos meus requisitos batem com as associações de um funcionário estrangeiro' — inferência, não dado direto.

Condições de explorabilidade: Conhecer o `EmployeeId` de outra empresa; privilégio `require-documents:view`.

ISO-04 **Informativa****BillPayment e TenantManagement: ledger de idempotência também é global (chave escolhida pelo cliente)**

server/Services/BillPayment/BillPayment.Infra/Idempotency/RequestManager.cs:15-16 (+ TenantManagement.Infra/Persistence/ClientRequest.cs:92, RequestManager.cs:77-84)

```
public Task<bool> ExistAsync(Guid id, CancellationToken ct)
    => db.ClientRequests.AnyAsync(r => r.Id == id, ct); // PK só em Id, sem TenantId
```

Por que é explorável: Mesmo desenho de ISO-03, mas aqui o controller gera um Guid novo quando o header vem vazio (EnsureRequestId), então não há o efeito Guid.Empty. Resta a colisão deliberada: um Guid já gasto por qualquer tenant faz o comando de outro tenant devolver resposta neutra sem executar. Exige adivinhar um Guid escolhido pelo cliente — não explorável na prática, mas cross-tenant por construção.

Condições de explorabilidade: Conhecer o x-requestid de outro tenant/usuário.

ISO-05 **Informativa****Nenhum DbContext tem HasQueryFilter global: o isolamento depende de cada query**

server/Services/BillPayment/BillPayment.Infra/Persistence/BillPaymentDbContext.cs:51-76 (grep HasQueryFilter vazio nos 3 BCs)

```
// BillPaymentDbContext.cs OnModelCreating: só remove FKs cross-aggregate; nenhum
HasQueryFilter(x => x.TenantId == ...)
// grep -rn HasQueryFilter server/Services -> 0 ocorrências
```

Por que é explorável: Não é falha por si — o BillPayment provou ponto a ponto que todas as queries filtram — mas significa que não há rede de segurança: uma query nova sem Where(TenantId) vazava dados silenciosamente. Os achados ISO-01/ISO-02 do PeopleManagement são exatamente esse tipo de deslize.

Condições de explorabilidade: Risco de regressão, não de exploração imediata.

3.2 Permissão definida no navegador

Todos os 30 gates de UI têm o escopo equivalente no servidor. Os achados desta categoria são superfícies do servidor que não passam pelo Keycloak ou que ficam expostas sem gating por ambiente.

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Média	server/Services/PeopleManagement/PeopleManagement.API/Program.cs:165-182 (+ appsettings.json:51-54; Filters/HangFireAuthorizationFilter.cs:8-12)	PRM-01 — Hangfire Dashboard fora do Keycloak, com Basic Auth de senha vazia por padrão e RequireSsl=false
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Controllers/TestController.cs:7-14 (+ ProtectedResourcePolicyProvider.cs:13-16 nos 3 BCs)	PRM-02 — Sem FallbackPolicy nas 3 APIs: action sem atributo é anônima — e o TestController do PeopleManagement está nessa condição em produção

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Program.cs:189-190	PRM-03 — PeopleManagement expõe Swagger/OpenAPI incondicionalmente em produção
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Program.cs:103-108 (+ BillPayment.API/Extension/CorsExtensions.cs:16-23; TenantManagement i dem)	PRM-04 — CORS AllowAnyOrigin no PeopleManagement; BillPayment/TenantManagement liberam qualquer origem com credenciais em Development
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Authentication/KeyclockExtesion.cs:13-14 (+ BillPayment KeyclockExtesion.cs:15; TenantManagement KeyclockExtesion.cs:13; appsettings.json "SslRequired": "none")	PRM-07 — RequireHttpsMetadata desligado por configuração (SslRequired: none) nas 3 APIs
Informativa	server/Services/PeopleManagement/PeopleManagement.API/Controllers/ArchiveController.cs:30 vs 12 (+ client/rufino_v2/.../document_group_with_templates_screen.dart:229; document_template_list_screen.dart:117)	PRM-05 — Nomes de recurso divergentes entre UI e servidor (e dentro do servidor)
Informativa	server/Services/TenantManagement/TenantManagement.API/Controllers/BaseController.cs:101-102 (+ PeopleManagement.API/Filters/HangFireAuthorizationFilter.cs:8-12)	PRM-06 — Código morto perigoso: fallback de identidade por header no TenantManagement e filtro allow-all do Hangfire

PRM-01

Média

Hangfire Dashboard fora do Keycloak, com Basic Auth de senha vazia por padrão e RequireSsl=false

server/Services/PeopleManagement/PeopleManagement.API/Program.cs:165-182 (+ appsettings.json:51-54; Filters/HangFireAuthorizationFilter.cs:8-12)

```
app.UseHangfireDashboard("/hangfire", new DashboardOptions
{
    Authorization = new[] { new BasicAuthAuthorizationFilter(new
BasicAuthAuthorizationFilterOptions
{
    RequireSsl = false,
    SslRedirect = false,
    LoginCaseSensitive = true,
    Users = new [] { new BasicAuthAuthorizationUser {
        Login = builder.Configuration["HangfireDashboard:Login"], // "admin" no
appsettings
        PasswordClear = builder.Configuration["HangfireDashboard:Password"] // "" no
appsettings
    } }
    }) }
});
// appsettings.json:51-54
"HangfireDashboard": { "Login": "admin", "Password": "" }
```

Por que é explorável: O dashboard permite disparar/apagar jobs (geração recorrente de documentos, refresh de webhook, lembretes WhatsApp) de todos os tenants e é montado antes de UseHttpsRedirection e fora do pipeline Keycloak. Login fixo `admin`; a senha depende exclusivamente da env var HangfireDashboard__Password — sem validação de startup. O HangFireAuthorizationFilter que retorna true incondicional existe no projeto (código morto) e é uma armadilha para quem trocar o filtro.

Condições de explorabilidade: Ambiente que não injete HangfireDashboard__Password (o workflow Azure injeta secretf:hangfire-password; nenhum docker-compose injeta). O comportamento da lib com PasswordClear vazio não foi verificado no fonte da biblioteca.

PRM-02 **Baixa****Sem FallbackPolicy nas 3 APIs: action sem atributo é anônima — e o TestController do PeopleManagement está nessa condição em produção**

server/Services/PeopleManagement/PeopleManagement.API/Controllers/TestController.cs:7-14 (+ ProtectedResourcePolicyProvider.cs:13-16 nos 3 BCs)

```
[Route("[controller]")]
public class TestController(ILogger<TestController> logger) : ControllerBase
{
    [HttpGet]
    public IActionResult GetTest()
    {
        return Ok("It's ok. v5");
    }
}
// ProtectedResourcePolicyProvider.cs:13-16
public Task<AuthorizationPolicy?> GetFallbackPolicyAsync()
=> Task.FromResult<AuthorizationPolicy?>(null);
```

Por que é explorável: GET /Test responde sem token; /Test/auth e /Test/auth/role usam o recurso fictício `teste` e não têm {company} (guard desligado). Nenhum Program.cs define FallbackPolicy = RequireAuthenticatedUser, então qualquer controller futuro sem atributo nasce público. No BillPayment o teste de erosão só cobre rotas que começam com api/v1 (EndpointProtectionTests.cs:40).

Condições de explorabilidade: Hoje só expõe string de versão; o risco é de regressão.

PRM-03 **Baixa****PeopleManagement expõe Swagger/OpenAPI incondicionalmente em produção**

server/Services/PeopleManagement/PeopleManagement.API/Program.cs:189-190

```
app.UseSwagger();
app.UseSwaggerUI();
```

Por que é explorável: Sem checagem de ambiente (BillPayment e TenantManagement só expõem em IsDevelopment()). Publica a superfície completa da API, incluindo o webhook e o TestController, para reconhecimento anônimo.

Condições de explorabilidade: Sempre ativo.

PRM-04 **Baixa****CORS AllowAnyOrigin no PeopleManagement; BillPayment/TenantManagement liberam qualquer origem com credenciais em Development**

server/Services/PeopleManagement/PeopleManagement.API/Program.cs:103-108 (+ BillPayment.API/Extension/CorsExtensions.cs:16-23; TenantManagement idem)

```
builder.Services.AddCors(options =>
{ options.AddPolicy("CorsPolicy", builder => builder
    .AllowAnyOrigin()
    .AllowAnyMethod()
    .AllowAnyHeader());
});
// CorsExtensions.cs:16-22 (BP/TM)
if (environment.IsDevelopment() && origins.Length == 0)
    policy.SetIsOriginAllowed(_ => true).AllowAnyHeader().AllowAnyMethod().AllowCredentials();
```

Por que é explorável: Com Bearer (sem cookies) o impacto é limitado: uma origem maliciosa só consegue chamar a API com um token que já possua. Ainda assim, AllowAnyOrigin retira a última barreira do navegador para exfiltração via XSS num cliente. Nos outros BCs o ramo de Development combina origem dinâmica com AllowCredentials — inofensivo apenas enquanto o compose ficar na máquina do desenvolvedor (docker-compose.yml:89 define Development).

Condições de explorabilidade: PeopleManagement: sempre. BP/TM: só ASPNETCORE_ENVIRONMENT=Development com Cors:AllowedOrigins vazio.

PRM-07 **Baixa****RequireHttpsMetadata desligado por configuração (SslRequired: none) nas 3 APIs**

server/Services/PeopleManagement/PeopleManagement.API/Authentication/KeycloakExtesion.cs:13-14 (+ Bill Payment KeycloakExtesion.cs:15; TenantManagement KeycloakExtesion.cs:13; appsettings.json "SslRequired": "none")

```
options.RequireHttpsMetadata =
string.IsNullOrEmpty(keycloakAuthenticationOptions.SslRequired) == false
    && keycloakAuthenticationOptions.SslRequired.Equals("external",
StringComparison.OrdinalIgnoreCase);
// appsettings.json: "SslRequired": "none" → RequireHttpsMetadata = false
```

Por que é explorável: Hoje inócuo porque AuthServerUrl é https://, mas um override Keycloak__AuthServerUrl=http://... (previsto no compose via KEYCLOAK_URL) passa a ser aceito sem aviso, permitindo obter o JWKS por canal não cifrado.

Condições de explorabilidade: Exige override de configuração para http.

PRM-05 **Informativa****Nomes de recurso divergentes entre UI e servidor (e dentro do servidor)**

server/Services/PeopleManagement/PeopleManagement.API/Controllers/ArchiveController.cs:30 vs 12 (+ client/rufino_v2/.../document_group_with_templates_screen.dart:229; document_template_list_screen.dart:117)

```
[ProtectedResource("archive", "edit")] // linha 12
[ProtectedResource("Archive", "edit")] // linha 30 – Keycloak distingue maiúsculas
// Flutter: resource: 'document_group' (servidor usa document-group);
hasPermission('company','edit') para editar template (servidor exige document-template:edit)
```

Por que é explorável: O servidor está correto em todos os casos (o gate real é o [ProtectedResource]); o efeito é funcional: botão sempre oculto ou 403 inesperado. Registrado porque divergência de nome é o que faz alguém 'consertar' afrouxando o servidor.

Condições de explorabilidade: Sem exploração.

PRM-06 **Informativa****Código morto perigoso: fallback de identidade por header no TenantManagement e filtro allow-all do Hangfire**

server/Services/TenantManagement/TenantManagement.API/Controllers/BaseController.cs:101-102 (+ PeopleManagement.API/Filters/HangFireAuthorizationFilter.cs:8-12)

```
protected Guid ResolveDecidingUserId(Guid headerUserId)
=> TryGetUserId(out var fromToken) ? fromToken : headerUserId; // nenhum controller chama
// HangFireAuthorizationFilter.cs:8-12
public bool Authorize(DashboardContext context) { return true; } // não referenciado
```

Por que é explorável: Nenhum dos dois está ligado ao pipeline hoje (grep confirma). O BillPayment removeu o mesmo fallback com justificativa explícita (ADR-007); manter a cópia no TenantManagement convida a reaproveitá-la.

Condições de explorabilidade: Sem exploração.

3.3 IDOR — referência direta a objeto sem checagem de posse

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Alta	server/Services/PeopleManagement/PeopleManagement.API/Controllers/CompanyController.cs:10, 40-42, 65-87 (+ CompanyQueries.cs:15,31,48; EditCompanyCommandHandler.cs:17-18)	IDR-01 — PeopleManagement: CompanyController sem {company} na rota — lê e edita qualquer empresa por Id livre

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Alta	server/Services/PeopleManagement/PeopleManagement.Services/Services/SignDocumentService.cs:117-118, 139-142, 155, 180-186 (+ DocumentController.cs:164-166; FileDownloadService.cs:22; ZapSignWebHookManagementService.cs:43-55,192-231)	IDR-02 — PeopleManagement: webhook ZapSign sem assinatura, sem escopo de tenant, com download de URL arbitrária (SSRF) e reenvio ao funcionário
Média	server/Services/BillPayment/BillPayment.Application/Models/Bills/BillModels.cs:16-38 (+ ImportBillCommand.cs:84-98; BillQueries.cs:170-171; S3AttachmentStorage.cs:145-149)	IDR-03 — BillPayment: POST /bills/import (JSON) aceita StorageKey, ContentHash, SourceId e ExternalMessageId do corpo sem validar pertencimento
Baixa	server/Services/BillPayment/BillPayment.Application/Expectations/Commands/RegisterBillExpectationCommand.cs:66	IDR-04 — BillPayment: HintSourceId do corpo não é validado contra o tenant em POST /expectations
Informativa	server/Services/TenantManagement/TenantManagement.Application/Models/Tenants/TenantModels.cs:10-20	IDR-05 — TenantManagement: operador pode escolher o TenantId no registro (backfill por desenho)

IDR-01 **Alta****PeopleManagement: CompanyController sem {company} na rota — lê e edita qualquer empresa por Id livre**

server/Services/PeopleManagement/PeopleManagement.API/Controllers/CompanyController.cs:10, 40-42, 65-87 (+ CompanyQueries.cs:15,31,48; EditCompanyCommandHandler.cs:17-18)

```
[Route("api/v1/[controller]")] // sem {company}: RouteAccessRequirement
concede em silêncio
public class CompanyController(...) : BaseController(logger)
{
    [HttpPut]
    [ProtectedResource("company", "edit")]
    public async Task<ActionResult<EditCompanyResponse>> EditCompany([FromBody]
    EditCompanyCommand request, ...)
    ...
    [HttpGet("complete")]
    [ProtectedResource("company", "view")]
    public async Task<ActionResult<CompanyDto>> GetCompaniesComplete([FromQuery] Guid id)
    // CompanyQueries.cs:48
    var query = _context.Companies.AsNoTracking().Where(x => x.Id == id);
    // EditCompanyCommandHandler.cs:17
    var company = await _companyRepository.FirstOrDefaultAsync(request.Id, ...)
```

Por que é explorável: O guard anti-IDOR só atua quando a rota tem o parâmetro `company`; aqui não há. Quem tiver company:view (todo usuário precisa dele para ver a própria empresa) chama GET /api/v1/company/complete?id= <GUID de outra empresa> e recebe CNPJ, e-mail, telefone e endereço completo; GET /list?id=a&id=b enumera em lote. Com company:edit, PUT /api/v1/company com Id alheio no corpo altera razão social, CNPJ e contato de qualquer empresa. GUIDs de empresa circulam no claim `companies` e em toda URL do produto. O próprio cliente Flutter depende deste comportamento (company_api_service.dart:28 passa os IDs do claim e o servidor devolve qualquer id pedido).

Condições de explorabilidade: Privilégio comum de tenant (company:view / company:edit) + GUID de outra empresa.

IDR-02 **Alta****PeopleManagement: webhook ZapSign sem assinatura, sem escopo de tenant, com download de URL arbitrária (SSRF) e reenvio ao funcionário**

server/Services/PeopleManagement/PeopleManagement.Services/Services/SignDocumentService.cs:117-118, 139-142, 155, 180-186 (+ DocumentController.cs:164-166; FileDownloadService.cs:22; ZapSignWebHookManagementService.cs:43-55,192-231)

```
// DocumentController.cs:164-166 – rota absoluta, sem {company}
[HttpPost("/api/v1/[controller]/webhook")]
[ProtectedResource("document", "webhook")]
public async Task<...> ReceiveWebhook([FromBody] JsonNode request, [FromHeader(Name = "x-requestid")] Guid requestId)

// SignDocumentService.cs:117 – lookup global por DocumentUnitId, sem CompanyId
var document = await _documentRepository.FirstOrDefaultAsync(x => x.DocumentsUnits.Any(x => x.Id == webhookEvent.DocumentUnitId), ...)
// :139-141 – baixa a URL do corpo e grava como 'documento assinado'
var primaryFile = await _fileDownloadService.DownloadFileFromUrlAsync(webhookEvent.Url, cancellationToken);
var primaryFileName = document.InsertUnitWithoutRequireValidation(webhookEvent.DocumentUnitId, Guid.NewGuid().ToString(), primaryFile.FileExtension);
// :180-186 – a mesma URL vai por WhatsApp ao funcionário
_whatsAppQueueService.EnqueueMediaMessage(phoneNumber: ..., media: url, fileName: fileName);
// FileDownloadService.cs:22
var response = await _httpClient.GetAsync(url, cancellationToken);
```

Por que é explorável: Não há verificação de HMAC/assinatura do provedor; a única barreira é o Keycloak conceder document#webhook ao token apresentado — e esse token é o client_credentials do client `zapsign` que o próprio sistema grava no cabeçalho do webhook cadastrado na ZapSign (ZapSignWebHookManagementService.cs:43-55), visível a quem acessa o painel ZapSign (ou a quem tiver o secret do client — ver SEC-01). Com ele: POST /api/v1/Document/webhook {event_type: doc_signed, external_id: <DocumentUnitId de qualquer tenant>, signed_file: http://169.254.169.254/... ou http://servico-interno/} faz o servidor baixar a URL (SSRF), gravar a resposta como documento assinado no blob da empresa alvo e enviar a URL ao celular do funcionário. doc_refused/doc_deleted/doc_expired invalida a unidade e toda a sessão de assinatura.

Condições de explorabilidade: Token com document#webhook (secret do client zapsign ou acesso ao painel ZapSign) + DocumentUnitId (GUID) da vítima. ISO-03 agrava: sem x-requestid, a partir do 2º evento os webhooks legítimos viram no-op.

IDR-03 **Média****BillPayment: POST /bills/import (JSON) aceita StorageKey, ContentHash, Sourcelid e ExternalMessageId do corpo sem validar pertencimento**

server/Services/BillPayment/BillPayment.Application/Models/Bills/BillModels.cs:16-38 (+ ImportBillCommand.cs:84-98; BillQueries.cs:170-171; S3AttachmentStorage.cs:145-149)

```
public sealed record ImportBillModel(
    string? DigtibleLine, string? PixPayload, string SourceKind,
    [property: JsonRequired] DateTime ReceivedAt,
    Guid? SourceId, string? SenderAddress, string? ExternalMessageId,
    string? ContentHash, string? StorageKey) // ← vêm do cliente
// ImportBillCommand.cs:84-85
var storageKey = document is null ? request.StorageKey : await storage.StoreAsync(...);
// S3AttachmentStorage.cs:145-149 – única barreira cross-tenant, e vira 500
if (!storageKey.StartsWith(Prefix(tenantId), StringComparison.Ordinal))
    throw new UnauthorizedAccessException("A chave de armazenamento não pertence a este tenant.");
```

Por que é explorável: A variante multipart (ImportBillFormModel, BillModels.cs:88-92) já zera ContentHash/StorageKey por reconhecer o risco; a JSON não. Um usuário com bill:import cria uma Bill cuja origem aponta para um Sourcelid alheio (gravado como 'prova de origem') e para qualquer StorageKey. Cross-tenant é barrado apenas pela checagem de prefixo (que sai como 500, confirmando a existência do objeto); intra-tenant nada barra: quem conhecer a StorageKey de um item ForeignPayer (documento de outro pagador lido pela caixa compartilhada, que o endpoint de quarentena recusa servir) o baixa por GET /bills/{id}/artifact. HintSourcelid em POST /expectations tem a mesma ausência de validação, sem vazamento (IDR-04).

Condições de explorabilidade: bill:import + bill:view; a StorageKey não sai em nenhum DTO (exige log/banco).

IDR-04 **Baixa****BillPayment: HintSourcelid do corpo não é validado contra o tenant em POST /expectations**

server/Services/BillPayment/BillPayment.Application/Expectations/Commands/RegisterBillExpectationCommand.cs:66

```
request.HintSourceId is { } sourceId ? CaptureSourceId.From(sourceId) : null, // sem
captureSources.GetAsync(tenantId, sourceId)
```

Por que é explorável: Payeeld no mesmo handler É validado (L48); HintSourcelid não. O único consumidor é ListByHintSourceAsync(tenantId, sourcelid) com filtro de tenant, logo um GUID alheio nunca casa — efeito é dado inconsistente no próprio tenant, não IDOR.

Condições de explorabilidade: expectation:manage.

IDR-05 **Informativa**

TenantManagement: operador pode escolher o TenantId no registro (backfill por desenho)

server/Services/TenantManagement/TenantManagement.Application/Models/Tenants/TenantModels.cs:10-20

```
public sealed record RegisterTenantModel(
    string Kind, string LegalName, string? TradeName, string PrimaryTaxId, string ContactEmail,
    string? ContactPhone, AddressInput Address, string OwnerEmail, IReadOnlyList<string>?
    Products,
    Guid? Id)    // ← vira o TenantId gravado no claim
```

Por que é explorável: Com tenant#create + tenant-access#edit (papel de plataforma tenant-admin) é possível emitir o claim tenants para um GUID arbitrário — inclusive um CompanyId de outro produto ainda não backfillado. A única defesa é ExistsAsync. É o caminho de backfill previsto; registrado para que o campo seja removido quando o backfill terminar.

Condições de explorabilidade: Restrito ao papel tenant-admin (back-office).

3.4 Chaves expostas (hardcode)

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Crítica	server/Services/PeopleManagement/PeopleManagement.API/appsettings.json (histórico) e client/rufino/secrets/config.json (histórico):commits 5b98cfff3 (2024-08-29), 6838d2be (2025-07-29), f109691d (2024-08-16), 3353f902 (2024-09-19)	SEC-01 — Segredos reais recuperáveis no histórico git (client secrets Keycloak, token ZapSign, secret do app)
Média	server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:9-10 (+ BillPayment.API/appsettings.json:10; TenantManagement.API/appsettings.json:10; server/docker-compose.yml:68,91,111,132,168,239,259; BillPayment/docker-compose.yml:12,52; TenantManagement/docker-compose.yml:12,26)	SEC-02 — Senha do Postgres embutida no appsettings.json base das 3 APIs e em todos os docker-compose
Média	server/docker-compose.yml:230-231, 239, 189-190, 135 (+ BillPayment/docker-compose.yml:15,73-74; portas 8082 e 8104 em :193,244; server/.env:13 a ponta 192.168.15.41)	SEC-03 — Keycloak local com admin Admin/Admin e MinIO com credenciais fracas, publicados no host
Média	azure/container-apps/evolution-api/template.env:8, 24, 38 (+ deploy.ps1:92; main.bicep:29-31; container-app.yaml:8)	SEC-04 — Evolution API (Azure): template.env com API key placeholder e CORS *, e deploy.ps1 injeta segredos como env vars em texto claro
Média	server/Services/PeopleManagement/PeopleManagement.Infra/Services/WhatsAppService.cs:40-47	SEC-05 — API key da Evolution API (WhatsApp) gravada em log Information a cada mensagem

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Authentication/AuthenticationExtension.cs:22 (+ BillPayment/TenantManagement AuthenticationExtension.cs:23; KeycloakAuthenticationOptions.cs Secret = string.Empty; Program.cs PM:176-177)	SEC-06 — Nenhuma API rejeita segredo vazio ao iniciar (sem ValidateOnStart/ValidateOptions)
Baixa	server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:13-14, 62	SEC-07 — Access Key ID do S3 (Garage) de produção e número de telefone reais versionados
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Controllers/BaseController.cs:24-49 (+ TenantManagement.API/Controllers/BaseController.cs:59-64; nenhum Command do TM implementa ISensitiveCommand)	SEC-08 — Payload completo dos Commands (CPF, filiação, endereço, dependentes, e-mails) em log Information
Informativa	server/Services/BillPayment/.env:4 (ignorado por .gitignore:29) – client/rufino_v2/secrets/GOOGLE_PLAY_SERVICE_ACCOUNT_JSON.json (ignorado por client/rufino_v2/.gitignore:72)	SEC-09 — Segredos reais presentes só em disco local (corretamente ignorados) e conta de serviço com manage-users amplo

SEC-01 **Crítica****Segredos reais recuperáveis no histórico git (client secrets Keycloak, token ZapSign, secret do app)**

server/Services/PeopleManagement/PeopleManagement.API/appsettings.json (histórico) e client/rufino/secrets/config.json (histórico):commits 5b98cff3 (2024-08-29), 6838d2be (2025-07-29), f109691d (2024-08-16), 3353f902 (2024-09-19)

```
git show 5b98cff3:server/Services/PeopleManagement/PeopleManagement.API/appsettings.json
 27: "secret": "uHuC...yU01"           (Keycloak client people-management-api, 32 chars)
git show 6838d2be:./appsettings.json
 33: "ClientSecret": "gJgE...qo0d"     (Keycloak client zapsign – o que autentica o webhook
IDR-02)
git show f109691d:./appsettings.json
 14: "Token": "Bearer 17ea...388e"     (API token ZapSign, 72 chars)
git show 3353f902:client/rufino/secrets/config.json
  5: "secret" : "ILnW...FBt9"         (client rufino-app, então confidencial)
remote: https://github.com/IgorCoura/rufino-project.git
```

Por que é explorável: Os quatro valores foram removidos da árvore entre 2024-08 e 2025-07, mas continuam a um `git show` de distância para qualquer pessoa com acesso ao repositório (e para qualquer clone/fork antigo). Se não foram rotacionados: o secret de people-management-api permite obter RPTs em nome do resource server; o secret de zapsign gera o token que o webhook aceita (IDR-02); o token ZapSign permite ler/assinar documentos na conta; o secret de rufino-app está mitigado porque o client virou público (realm-import-2026-08-18.json:2019). Não há evidência no repositório de rotação.

Condições de explorabilidade: Acesso de leitura ao repositório GitHub (visibilidade não verificada) ou a qualquer clone antigo. Crítica enquanto não houver confirmação de rotação.

SEC-02 Média

Senha do Postgres embutida no appsettings.json base das 3 APIs e em todos os docker-compose

server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:9-10 (+ BillPayment.API/appsettings.json:10; TenantManagement.API/appsettings.json:10; server/docker-compose.yml:68,91,111,132,168,239,259; BillPayment/docker-compose.yml:12,52; TenantManagement/docker-compose.yml:12,26)

```
"ConnectionStrings": {
  "Postgresql":
  "Server=peoplemanagement.db;Port=5432;Database=PeopleManagementDb;SearchPath=people_management;UserId=admin;Password=5up3rS3nh@",
  "HangfireConnection":
  "Server=peoplemanagement.db;Port=5432;Database=hangfiredb;UserId=admin;Password=5up3rS3nh@"
}
// docker-compose.yml:68 POSTGRES_PASSWORD: "5up3rS3nh@" (mesma senha desde 2022, em 9 lugares)
```

Por que é explorável: É default de desenvolvimento, mas está no appsettings BASE (não no Development): qualquer ambiente que não sobrescreva ConnectionStrings__X sobe com esta senha, e nenhuma API valida a configuração ao iniciar. Os composes publicam os bancos em 0.0.0.0 (8042/8102/8112/8081) — na LAN, qualquer máquina entra como admin. A senha também é lida no PDF gerado a partir de appsettings.json via XSS-01.

Condições de explorabilidade: Compose exposto além da máquina do desenvolvedor, ou ambiente sem override de env var. Produção (Azure/Dokploy) injeta secretref/env.

SEC-03 Média

Keycloak local com admin Admin/Admin e MinIO com credenciais fracas, publicados no host

server/docker-compose.yml:230-231, 239, 189-190, 135 (+ BillPayment/docker-compose.yml:15,73-74; portas 8082 e 8104 em :193,244; server/.env:13 aponta 192.168.15.41)

```
KC_BOOTSTRAP_ADMIN_USERNAME: Admin
KC_BOOTSTRAP_ADMIN_PASSWORD: Admin
KC_DB_PASSWORD: 5up3rS3nh@
...
MINIO_ROOT_USER: "billpayment"
MINIO_ROOT_PASSWORD: "billpayment-dev"
Storage__SecretKey: "billpayment-dev"
```

Por que é explorável: O Keycloak local importa o realm `rufino` completo (clients, usuários, SMTP mascarado) e fica em 0.0.0.0:8082 — com Admin/Admin qualquer máquina da LAN vira administrador do realm e pode emitir tokens aceitos pelas APIs locais. O .env do servidor referencia o IP de LAN 192.168.15.41, indicando que o compose é acessado por outras máquinas.

Condições de explorabilidade: Apenas ambiente de desenvolvimento via compose; agrava se a mesma máquina hospedar dados reais (seed com CNPJ/CPF reais existe em seed-tenant.local.json, ignorado).

SEC-04 Média

Evolution API (Azure): template.env com API key placeholder e CORS *, e deploy.ps1 injeta segredos como env vars em texto claro

azure/container-apps/evolution-api/template.env:8, 24, 38 (+ deploy.ps1:92; main.bicep:29-31; container-app.yaml:8)

```
AUTHENTICATION_API_KEY=senha-aqui
CORS_ORIGIN=*
DATABASE_CONNECTION_URI=postgresql://user:password@evolution.db:5432/evolution_api
# deploy.ps1:92
$azCommand += '--env-vars'      # lê o .env inteiro e passa como variáveis comuns, não --
secrets/secretref
```

Por que é explorável: A Evolution API é o gateway de WhatsApp da plataforma e tem ingress externo (main.bicep:29-31). Se alguém copiar template.env → .env sem trocar a chave, a API sobe aceitando `senha-aqui` como chave global com CORS aberto. Mesmo com chave forte, o deploy.ps1 a grava como variável de ambiente comum do Container App (visível no portal/az CLI) em vez de secretref — o main.bicep até marca environmentVariables como @secure(), mas o script não usa o bicep.

Condições de explorabilidade: Operador que não substitua o placeholder; exposição da chave a quem lê a configuração do Container App.

SEC-05 Média

API key da Evolution API (WhatsApp) gravada em log Information a cada mensagem

server/Services/PeopleManagement/PeopleManagement.Infra/Services/WhatsAppService.cs:40-47

```
var apiKeyMasked = _options.ApiKey;      // nome diz 'masked', valor é a chave inteira
_logger.LogInformation(
    "Sending WhatsApp message - URL: {FullUrl}, Instance: {Instance}, ApiKey: {ApiKey}, Body: {Body}",
    fullUrl, _options.Instance, apiKeyMasked, bodyJson);
```

Por que é explorável: A chave global da Evolution API (que permite enviar mensagens em nome da instância e ler conversas) vai para o sink de log em texto claro junto com número de telefone e corpo da mensagem (PII). Quem lê logs (Azure Log Analytics, contêiner) obtém a credencial.

Condições de explorabilidade: Nível de log Information ligado (padrão em appsettings e no workflow de deploy).

SEC-06 **Baixa****Nenhuma API rejeita segredo vazio ao iniciar (sem ValidateOnStart/IValidateOptions)**

server/Services/PeopleManagement/PeopleManagement.API/Authentication/AuthenticationExtesion.cs:22 (+ BillPayment/TenantManagement AuthenticationExtesion.cs:23; KeycloakAuthenticationOptions.cs Secret = string.Empty; Program.cs PM:176-177)

```
configuration.GetSection(KeycloakAuthenticationOptions.Section).Bind(keycloakAuthenticationOptions); // só Bind; Secret vazio passa
// grep ValidateOnStart|ValidateDataAnnotations|IValidateOptions → 0 ocorrências nas 3 APIs
```

Por que é explorável: Keycloak:Credentials:secret, HangfireDashboard:Password, ConnectionStrings, S3, ZapSign e WhatsApp são aceitos vazios; a falha só aparece na primeira permissão avaliada ou no primeiro envio. O BillPayment mitiga com o padrão 'desligado falha alto' (UnconfiguredSecretVault/Storage), mas isso não cobre o secret do Keycloak. Um deploy com env var faltando sobe 'saudável' com os defaults de SEC-02/PRM-01.

Condições de explorabilidade: Erro operacional de configuração.

SEC-07 **Baixa****Access Key ID do S3 (Garage) de produção e número de telefone reais versionados**

server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:13-14, 62

```
"S3": {
  "ServiceURL": "https://garage.couratechsafety.cloud",
  "AccessKey": "GKcf...e931",
  "SecretKey": "",
  ...
  "WhatsApp": { ..., "HealthCheckNumber": "5511968608425" }
```

Por que é explorável: O Access Key ID sozinho não autentica (SecretKey vazia), mas é metade da credencial de produção e revela o endpoint do storage; o telefone é PII de pessoa real. Ambos deveriam viver em configuração de ambiente.

Condições de explorabilidade: Precisa da SecretKey para explorar.

SEC-08 **Baixa****Payload completo dos Commands (CPF, filiação, endereço, dependentes, e-mails) em log Information**

server/Services/PeopleManagement/PeopleManagement.API/Controllers/BaseController.cs:24-49 (+ TenantManagement.API/Controllers/BaseController.cs:59-64; nenhum Command do TM implementa ISensitiveCommand)

```
_logger.LogInformation(
    "----- Sending command: {CommandName} - {IdProperty}: {CommandId} ({@Command}) - RequestId : {RequestId} -----",
    CommandName, IdProperty, CommandId, Command, RequestId); // {@Command} desestrutura o objeto inteiro
```

Por que é explorável: AlterIdCard, AlterPersonalInfo, CreateDependent, RegisterTenant etc. vão inteiros para o log, duas vezes (envio e resultado). O BillPayment resolveu isso com ISensitiveCommand + omissão do payload no resultado; o PeopleManagement e o TenantManagement não. É exposição de dado pessoal (LGPD) a quem lê logs, não credencial.

Condições de explorabilidade: Log Information ligado.

SEC-09 **Informativa****Segredos reais presentes só em disco local (corretamente ignorados) e conta de serviço com manage-users amplo**

server/Services/BillPayment/.env:4 (ignorado por .gitignore:29) – client/rufino_v2/secrets/GOOGLE_PLAY_SERVICE_ACCOUNT_JSON.json (ignorado por client/rufino_v2/.gitignore:72)

```
BILLPAYMENT_MASTER_KEY=3goR...PkM=          # master key AES-256-GCM do cofre de tenant_secrets (não rastreado)
GOOGLE_PLAY_SERVICE_ACCOUNT_JSON.json        # private_key RSA da conta de publicação (não rastreado)
# realm export: service-account-tenant-management-provisioner → realm-management: view-users, manage-users
```

Por que é explorável: Confirmado por git ls-files/check-ignore que nenhum deles foi versionado. Registrado porque (a) a máquina de desenvolvimento guarda a chave que cifra todas as credenciais de tenant e a chave que publica o app na Play Store, e (b) a conta de serviço do provisionador tem manage-users sobre o realm inteiro, então um vazamento de TenantProvisioning__ClientSecret dá reset de senha de qualquer usuário.

Condições de explorabilidade: Sem exposição no repositório.

3.5 Inputs sem tratamento (XSS / HTML ativo)

No frontend a categoria não se aplica na forma clássica (nenhum innerHTML/eval; o único renderizador de HTML é o HtmlWidget do corpo de e-mail, que não executa script). O risco real está no servidor: HTML controlado pelo tenant executado em Chromium com JavaScript, origem file:// e sem sandbox.

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
Crítica	server/Services/PeopleManagement/PeopleManagement.Infra/Services/PdfService.cs:94-96, 149-162 (+ BrowserProvider.cs:17-29; DocumentTemplateController.cs:50-53; LocalStorageService.cs:19-20; FileName.cs:20-31)	XSS-01 — Template HTML enviado pelo tenant é renderizado em Chromium com JavaScript, origem file:// e sem sandbox — leitura de arquivos do container e SSRF
Alta	server/Services/PeopleManagement/PeopleManagement.Infra/Services/HtmlService.cs:93-105 (+ DocumentService.cs:362,409,444-469)	XSS-02 — Valores de funcionário/empresa interpolados no HTML do documento sem encoding — mesmo efeito de XSS-01 com privilégio menor
Baixa	client/rufino_v2/nginx/default.conf:9-17 (+ client/rufino_v2/web/index.html:41-43; grep X-Frame-Options Content-Security-Policy UseHsts nas 3 APIs → 0)	XSS-03 — Sem cabeçalhos de segurança/CSP no nginx do Flutter web nem nas APIs; pdf.js carregado de CDN sem integrity
Baixa	server/Services/PeopleManagement/PeopleManagement.API/Filters/ApplicationExceptionHandler.cs:44 (+ 16-37)	XSS-04 — PeopleManagement devolve a mensagem interna do Postgres ao cliente em erros de banco
Baixa	server/Services/PeopleManagement/PeopleManagement.Infra/Services/LocalStorageService.cs:19-20 (+ 51-57; DocumentTemplateController.cs:52 limita só o request a 12 MB)	XSS-05 — Zip de template extraído sem limite de tamanho descompactado (zip bomb) e diretório antigo nunca apagado

XSS-01 **Crítica****Template HTML enviado pelo tenant é renderizado em Chromium com JavaScript, origem file:// e sem sandbox — leitura de arquivos do container e SSRF**

server/Services/PeopleManagement/PeopleManagement.Infra/Services/PdfService.cs:94-96, 149-162 (+ BrowserProvider.cs:17-29; DocumentTemplateController.cs:50-53; LocalStorageService.cs:19-20; FileName.cs:20-31)

```
// PdfService.cs:94-96
await using var page = await browser.NewPageAsync();
await page.GoToAsync("file://" + indexHtmlPath); // origem file:// para resolver CSS relativo
await page.SetContentAsync(processedBody); // HTML do zip enviado pelo tenant
// BrowserProvider.cs:17-29
private static readonly string[] ChromeArgs = [ "--no-sandbox", "--disable-setuid-sandbox", "--disable-gpu", ... ];
// grep SetJavaScriptEnabled|SetRequestInterception → 0 ocorrências
// PdfService.cs:154-158 – href de <link rel=stylesheet> lido do disco sem canonicalizar
var cssPath = Path.Combine(templateDirectory, href.Replace('/', Path.DirectorySeparatorChar));
if (File.Exists(cssPath)) { var css = File.ReadAllText(cssPath); return $"<style>{css}</style>"; }
}
```

Por que é explorável: POST /api/v1/{company}/documenttemplate/Upload (document-template:upload) extrai um zip para app_files/templates e o index.html vira o conteúdo da página. Com JavaScript ligado e origem file://, <iframe src="file:///app/appsettings.json">` é impresso no PDF devolvido por GET document/generate/... (lê a connection string de SEC-02, /proc/self/environ com Keycloak\_\_Credentials\_\_secret, etc.); `/^fetch()` alcançam metadados de nuvem e serviços internos (SSRF). Sem <script>, o InlineExternalStyles já lê qualquer arquivo por <link rel="stylesheet" href="..../appsettings.json">` (o VO FileName aceita `../x.html`). --no-sandbox amplia o impacto de qualquer bug de renderização para o processo do container. Header/footer do zip seguem o mesmo caminho.

Condições de explorabilidade: Admin de tenant com document-template:upload + document:generate. Sem sanitização (nenhum HtmlSanitizer/AngleSharp nos .csproj).

XSS-02 **Alta****Valores de funcionário/empresa interpolados no HTML do documento sem encoding — mesmo efeito de XSS-01 com privilégio menor**

server/Services/PeopleManagement/PeopleManagement.Infra/Services/HtmlService.cs:93-105 (+ DocumentService.cs:362,409,444-469)

```
private static string ReplaceDoubleBraces(string content, JsonNode? values, bool keepIfMissing = false)
{
    Regex regex = HtmlParamRegex();
    return regex.Replace(content, m =>
    {
        string[] keys = m.Groups[1].Value.Split('.');
        var result = GetValueFromJson(keys, values)?.ToString();
        if (result != null)
            return result;          // sem HtmlEncoder / WebUtility.HtmlEncode
        return keepIfMissing ? m.Value : string.Empty;
    });
}
```

Por que é explorável: Os `values` são o JSON montado por RecoverInfoToDocument a partir de nome, endereço, dependentes, cargo, empresa etc. Quem tem employee:edit (RH comum, privilégio menor que upload de template) grava ``<iframe src=file:///app/appsettings.json>`` ou ``<script>`` num campo de nome/complemento; no próximo documento gerado para esse funcionário o HTML é executado exatamente como em XSS-01. Também vale para dados de empresa (CompanyController, ver IDR-01) e de Role de outra empresa (ISO-01).

Condições de explorabilidade: employee:edit (ou company:edit) + geração de documento (automática por jobs recorrentes ou por qualquer usuário com document:generate).

XSS-03 **Baixa****Sem cabeçalhos de segurança/CSP no nginx do Flutter web nem nas APIs; pdf.js carregado de CDN sem integrity**

client/rufino_v2/nginx/default.conf:9-17 (+ client/rufino_v2/web/index.html:41-43; grep X-Frame-Options|Content-Security-Policy|UseHsts nas 3 APIs → 0)

```
add_header Cache-Control "no-cache, no-store, must-revalidate" always; # únicos add_header do
arquivo
# index.html:41
<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/2.11.338/pdf.min.js"></script> # sem
integrity/crossorigin
```

Por que é explorável: Defesa em profundidade: sem CSP/X-Frame-Options/nosniff/HSTS o app web fica sem a segunda barreira caso surja um vetor de injeção, e o pdf.js de CDN sem SRI executa qualquer código que o CDN sirva. Não há vetor de XSS no Flutter hoje (ver pontos fortes).

Condições de explorabilidade: Comprometimento do CDN ou vetor futuro de injeção.

XSS-04 **Baixa****PeopleManagement devolve a mensagem interna do Postgres ao cliente em erros de banco**

server/Services/PeopleManagement/PeopleManagement.API/Filters/ApplicationExceptionHandler.cs:44 (+ 16-37)

```
var message = exception.InnerException?.Message; // Npgsql: nomes de tabela/coluna/constraint → JSON 400 "Errors"
```

Por que é explorável: Vazamento de informação (esquema do banco) em JSON — não é XSS porque nada é renderizado como HTML, mas facilita reconhecimento.

BillPayment/TenantManagement devolvem só erros de domínio.

Condições de explorabilidade: Qualquer requisição que viole constraint.

XSS-05 **Baixa****Zip de template extraído sem limite de tamanho descompactado (zip bomb) e diretório antigo nunca apagado**

server/Services/PeopleManagement/PeopleManagement.Infra/Services/LocalStorageService.cs:19-20 (+ 51-57; DocumentTemplateController.cs:52 limita só o request a 12 MB)

```
using var zipArchive = new ZipArchive(stream);
zipArchive.ExtractToDirectory(destinationDirectoryPath, overwriteFiles: true); // sem checar tamanho total das entradas
```

Por que é explorável: 12 MB de zip podem virar gigabytes em disco. ExtractToDirectory do .NET 8 bloqueia zip-slip (verificado), então não há traversal na extração — o traversal fica no href de CSS (XSS-01). DeleteAsync combina os argumentos invertidos e não remove o diretório anterior, acumulando templates antigos.

Condições de explorabilidade: document-template:upload.

4. Recomendações priorizadas

P1

Isolar a geração de PDF e encodar tudo que entra no HTML

Desligar JavaScript (`page.SetJavaScriptEnabledAsync(false)`), interceptar requests e bloquear tudo que não seja `data:/o` próprio template, não navegar para `file://` (inlinar CSS lendo só arquivos canonicalizados dentro do diretório do template), rodar o Chromium com sandbox num container separado sem acesso à rede interna e sem appsettings/segedos, e aplicar `HtmlEncoder.Default.Encode` em `ReplaceDoubleBraces`. Rejeitar `FileName` com separadores de caminho.

Achados: XSS-01, XSS-02, XSS-05

P1

Rotacionar os segredos expostos no histórico e decidir sobre reescrita

Regenerar os client secrets `people-management-api` e `zapsign` no Keycloak, o token da API `ZapSign` e (por precaução) confirmar que `rufino-app` continua público. Verificar visibilidade do repositório; se público ou com forks, rodar `git filter-repo` após a rotação. Ativar `secret scanning/push protection` no GitHub.

Achados: SEC-01

P1

Trazer `CompanyController` e o webhook `ZapSign` para dentro do guard de tenant

Mover `Company` para `api/v1/{company}/company` (ou filtrar pelos IDs do claim ``companies`` no servidor) e validar `Id` do corpo contra a rota. No webhook: validar assinatura/segreto da `ZapSign`, restringir `signed_file` a hosts da `ZapSign` (`allowlist`) e ao `Content-Type` `application/pdf`, resolver a `DocumentUnit` junto com a `CompanyId` gravada na sessão de assinatura, não reenviar a URL externa ao funcionário (enviar o blob), e exigir `x-requestid` próprio (ver P2 idempotência).

Achados: IDR-01, IDR-02

P2

Fechar o isolamento residual do `PeopleManagement`

Validar todo ID relacionado do corpo com `ExistsAsync(id, companyId)` antes de atribuir (`RoleId`, `WorkPlaceld`, `DepartmentId`, `PositionId`, `DocumentGroupId`, `DocumentsTemplatesIds`); adicionar `CompanyId` aos joins/subqueries listados; escopar o ledger de idempotência por (`CompanyId`, `RequestId`) e recusar `Guid.Empty` (`[BindRequired]` ou `EnsureRequestId` como no `BillPayment`). Avaliar `HasQueryFilter` por tenant como rede de segurança nos 3 `DbContexts`.

Achados: ISO-01, ISO-02, ISO-03, ISO-04, ISO-05

P2

Blindar o convite de tenant contra troca de e-mail

No provisionador, só gravar claims em usuário com `emailVerified=true` (ou criar convite pendente resolvido pelo sub após verificação); no realm, remover ``user`` de `permissions.edit` do atributo `email` ou ligar `verifyEmail`; em `/me/tenants`, chavear por `IdentityUserId` (sub) em vez de e-mail.

Achados: ISO-06

P2

Validar segredos no startup e remover defaults de credencial da árvore

Adicionar `OptionsBuilder.Validate(...).ValidateOnStart()` para `Keycloak:Credentials:secret`, `HangfireDashboard:Password` (quando `!IsDevelopment`), `connection strings` e chaves de integração; mover a senha do Postgres e o `AccessKey` do Garage para `appsettings.Development.json/user-secrets`; trocar `5up3rS3nh@/Admin` nos composes por `${VAR:?}` lido do `.env` ignorado e não publicar portas em `0.0.0.0`; no `deploy.ps1` da `Evolution API` usar `--secrets + secretref` e trocar `CORS_ORIGIN=*`.

Achados: SEC-02, SEC-03, SEC-04, SEC-06, SEC-07, PRM-01

P2**Tirar segredos e PII dos logs**

Remover ApiKey do LogInformation do WhatsAppService; adotar o padrão ISensitiveCommand/omissão de payload do BillPayment no PeopleManagement e no TenantManagement; não repetir {@Command} no log de resultado.

Achados: SEC-05, SEC-08

P2**Endurecer a importação manual do BillPayment**

Fazer ImportBillModel (JSON) zerar StorageKey/ContentHash como o multipart já faz; validar Sourceld (e HintSourceld) com ICaptureSourceRepository.GetAsync(tenantId, ...) quando informado, ou recusá-los fora de ManualUpload; mapear UnauthorizedAccessException do storage para 404.

Achados: IDR-03, IDR-04

P3**Hardening do PeopleManagement e do Hangfire**

Proteger /hangfire com a policy do Keycloak (ou IP allowlist + RequireSsl=true), apagar HangFireAuthorizationFilter; remover TestController; condicionar Swagger a IsDevelopment; CORS com WithOrigins; FallbackPolicy = RequireAuthenticatedUser nos 3 Program.cs; RequireHttpsMetadata=true fora de dev; não devolver InnerException.Message; adicionar cabeçalhos de segurança no nginx e SRI no pdf.js.

Achados: PRM-02, PRM-03, PRM-04, PRM-07, XSS-03, XSS-04

P3**Limpeza de código morto e nomes divergentes**

Remover ResolveDecidingUserId(Guid headerUserId) do TenantManagement; unificar 'Archive'/'archive'; corrigir os gates do Flutter (document_group → document-group, company:edit → document-template:edit); remover RegisterTenantModel.Id ao fim do backfill; ampliar o teste de erosão do BillPayment para toda rota (não só api/v1).

Achados: PRM-05, PRM-06, IDR-05

5. Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Os delimitadores `---` `ISSUE n` `---` / `---` `FIM ISSUE n` `---` não fazem parte da issue. Achados triviais do mesmo tema foram agrupados.

--- ISSUE 1 ---

```
# [Segurança] Geração de PDF executa HTML/JS do tenant em Chromium sem isolamento (LFI/SSRF)

**Labels sugeridas:** `security`, `severity:critical`, `people-management`

## Problema
O PeopleManagement gera documentos renderizando, em um Chromium headless (PuppeteerSharp) lançado com
`--no-sandbox`, o `index.html` do zip enviado pelo tenant em `POST
/api/v1/{company}/documenttemplate/Upload`, depois de interpolar nele – sem nenhum encoding – os dados
do funcionário/empresa. A página é aberta com origem `file://` (`GoToAsync("file://" +
indexHtmlPath)`) e JavaScript fica ligado; não há interceptação de requests. Além disso,
`InlineExternalStyles` lê do disco qualquer `href` de `

```

```

false)
{
    Regex regex = HtmlParamRegex();
    return regex.Replace(content, m =>
    {
        string[] keys = m.Groups[1].Value.Split('.');
        var result = GetValueFromJson(keys, values)?.ToString();
        if (result != null)
            return result; // sem HtmlEncoder / WebUtility.HtmlEncode
        return keepIfMissing ? m.Value : string.Empty;
    });
}
...

```

Os `values` são o JSON montado por RecoverInfoToDocument a partir de nome, endereço, dependentes, cargo, empresa etc. Quem tem employee:edit (RH comum, privilégio menor que upload de template) grava ``<iframe src=file:///app/appsettings.json>`` ou ``<script>`` num campo de nome/complemento; no próximo documento gerado para esse funcionário o HTML é executado exatamente como em XSS-01. Também vale para dados de empresa (CompanyController, ver IDR-01) e de Role de outra empresa (ISO-01).

XSS-05 – Zip de template extraído sem limite de tamanho descompactado (zip bomb) e diretório antigo nunca apagado

`server/Services/PeopleManagement/PeopleManagement.Infra/Services/LocalStorageService.cs:19-20 (+ 51-57; DocumentTemplateController.cs:52 limita só o request a 12 MB)`

```

```csharp
using var zipArchive = new ZipArchive(stream);
zipArchive.ExtractToDirectory(destinationDirectoryPath, overwriteFiles: true); // sem checar tamanho
total das entradas
...

```

12 MB de zip podem virar gigabytes em disco. ExtractToDirectory do .NET 8 bloqueia zip-slip (verificado), então não há traversal na extração – o traversal fica no href de CSS (XSS-01). DeleteAsync combina os argumentos invertidos e não remove o diretório anterior, acumulando templates antigos.

#### ## Impacto

Leitura de segredos do container (connection string, secret do Keycloak, tokens de integração) e SSRF para serviços internos a partir de um privilégio comum de tenant (`employee:edit`) ou de admin de tenant (upload de template). Com `--no-sandbox`, qualquer bug de renderização do Chromium vira execução no processo do container. Cruza a fronteira do tenant para a infraestrutura da plataforma.

#### ## Sugestão de correção

1. Aplicar `HtmlEncoder.Default.Encode(result)` em `HtmlService.ReplaceDoubleBraces` (e nos demais pontos que devolvem valor do JSON) – dados nunca viram HTML ativo.
1. Em `PdfService`: `await page.SetJavaScriptEnabledAsync(false);` `SetRequestInterceptionAsync(true)` abortando tudo que não seja `data:`/`about:blank`; não usar `GoToAsync("file:///...")` – montar o HTML já com o CSS inlineado e chamar `SetContentAsync` numa página `about:blank`.
1. Em `InlineExternalStyles`: resolver `Path.GetFullPath(Path.Combine(templateDirectory, href))` e recusar se não começar com o `templateDirectory` canonicalizado; recusar `href` absoluto ou com `..`. No VO `FileName`, recusar qualquer separador de caminho.
1. Lançar o Chromium com sandbox (usuário não-root no Dockerfile, sem `--no-sandbox`) – idealmente num container/serviço dedicado de renderização, sem `appsettings`/segredos montados e com egress de rede bloqueado.
1. Limitar o tamanho total descompactado do zip de template (somar `entry.Length` antes de extrair; rejeitar acima de N MB) e corrigir `LocalStorageService.DeleteAsync` para apagar o diretório anterior.

#### ## Critérios de aceite

- [ ] Template com ``<script>document.body.innerHTML='pwned'</script>`` gera PDF sem executar o script (conteúdo original preservado).
- [ ] Template com ``<iframe src="file:///app/appsettings.json">`` e com ``<link rel="stylesheet" href="../../appsettings.json">`` gera PDF sem o conteúdo do arquivo (teste de integração cobrindo os dois vetores).
- [ ] Template com ```` não abre conexão (request interceptado/abortado;

verificável por log do interceptor ou listener local).

- [ ] Funcionário com nome `<b>x</b><iframe src=file:///etc/passwd>` aparece no PDF literalmente como texto, sem tag interpretada.
- [ ] `BrowserProvider.ChromeArgs` não contém `--no-sandbox`/`--disable-setuid-sandbox` e o container roda como usuário não-root.
- [ ] Upload de zip cujo conteúdo descompactado excede o limite configurado retorna 400 sem escrever em disco.
- [ ] CLAUDE.md do PeopleManagement documenta o modelo de renderização segura (JS desligado, sem file://, limite de zip).

### --- FIM ISSUE 1 ---

### --- ISSUE 2 ---

# [Segurança] Rotacionar segredos expostos no histórico do repositório (Keycloak, ZapSign)

**Labels sugeridas:** ``security`, `severity:critical`, `secrets`, `ops``

#### ## Problema

Quatro credenciais reais foram commitadas entre 2024-08 e 2025-07 e, embora já removidas da árvore, continuam recuperáveis com `git show` em qualquer clone do repositório `github.com/IgorCoura/rufino-project`: o client secret do resource server `people-management-api` (commit 5b98cff3), o client secret do client `zapsign` (6838d2be) – que é justamente o que autentica o webhook de assinatura –, o token da API ZapSign (f109691d) e o secret do client `rufino-app` (3353f902, hoje mitigado porque o client virou público). Não há evidência no repositório de que tenham sido rotacionados.

#### ## Evidência

### SEC-01 – Segredos reais recuperáveis no histórico git (client secrets Keycloak, token ZapSign, secret do app)  
`server/Services/PeopleManagement/PeopleManagement.API/appsettings.json` (histórico) e `client/rufino/secrets/config.json` (histórico): commits 5b98cff3 (2024-08-29), 6838d2be (2025-07-29), f109691d (2024-08-16), 3353f902 (2024-09-19)

```text

```
git show 5b98cff3:server/Services/PeopleManagement/PeopleManagement.API/appsettings.json
27: "secret": "uhuC...yU01" (Keycloak client people-management-api, 32 chars)
git show 6838d2be:client/zapsign/appsettings.json
33: "ClientSecret": "gJgE...qo0d" (Keycloak client zapsign – o que autentica o webhook IDR-02)
git show f109691d:client/zapsign/appsettings.json
14: "Token": "Bearer 17ea...388e" (API token ZapSign, 72 chars)
git show 3353f902:client/rufino/secrets/config.json
5: "secret": "ILnW...FBt9" (client rufino-app, então confidencial)
remote: https://github.com/IgorCoura/rufino-project.git
```
```

Os quatro valores foram removidos da árvore entre 2024-08 e 2025-07, mas continuam a um `git show` de distância para qualquer pessoa com acesso ao repositório (e para qualquer clone/fork antigo). Se não foram rotacionados: o secret de `people-management-api` permite obter RPTs em nome do resource server; o secret de `zapsign` gera o token que o webhook aceita (IDR-02); o token ZapSign permite ler/assinar documentos na conta; o secret de `rufino-app` está mitigado porque o client virou público (`realm-import-2026-08-18.json:2019`). Não há evidência no repositório de rotação.

#### ## Impacto

Com o secret de `zapsign` obtém-se um token com `document#webhook` e explora-se a issue do webhook (forjar assinaturas, SSRF). Com o token ZapSign lê-se/assina-se documentos na conta da empresa. Com o secret de `people-management-api` obtém-se RPTs em nome do resource server. Qualquer pessoa com acesso de leitura ao repositório (ou a um fork/clone antigo) possui esses valores.

#### ## Sugestão de correção

1. No Keycloak (realm rufino): regenerar o secret dos clients `people-management-api` e `zapsign`; atualizar `secretref:keycloak-secret` e `secretref:authorization-clientsecret` no Container App /

Dokploy.

1. Na ZapSign: revogar o token de API antigo e emitir um novo; atualizar `secretref:zapsign-token`.
1. Confirmar que `rufino-app` está `publicClient: true` no realm de produção (o export de 2026-08-18 mostra isso) – se ainda for confidencial, regenerar.
1. Verificar a visibilidade do repositório e forks; se público ou com forks externos, após a rotação rodar `git filter-repo` removendo os caminhos afetados e forçar push (coordenar com todos os clones).
1. Ativar GitHub secret scanning + push protection e adicionar um hook de pre-commit (gitleaks) ao repositório.
1. Registrar a rotação (data, quem) num ADR ou no CHANGELOG de segurança.

## Critérios de aceite

- [ ] Os quatro valores antigos não autenticam mais (teste manual: token endpoint do Keycloak com o secret antigo retorna `invalid\_client`; API ZapSign retorna 401 com o token antigo).
- [ ] Serviços em produção operam com os novos valores (webhook ZapSign recebido com sucesso após a rotação; permissões UMA avaliadas sem erro).
- [ ] Secret scanning/push protection habilitados no repositório GitHub.
- [ ] Decisão sobre reescrita do histórico registrada (feita ou explicitamente descartada com justificativa).

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

# [Segurança] Webhook ZapSign aceita qualquer DocumentUnitId e baixa URL arbitrária (SSRF / forja de assinatura)

**Labels sugeridas:** `security`, `severity:high`, `people-management`, `idor`

## Problema

`POST /api/v1/Document/webhook` usa rota absoluta sem `{company}`, então o guard de tenant concede em silêncio; a única barreira é o Keycloak conceder `document#webhook` ao token – o mesmo token `client\_credentials` do client `zapsign` que o sistema grava no cabeçalho do webhook cadastrado na ZapSign (visível no painel do provedor) e cujo secret está no histórico git. Não há verificação de assinatura/HMAC. O handler localiza a `DocumentUnit` globalmente por `external\_id`, baixa `signed\_file` de qualquer URL com um `HttpClient` sem allowlist, grava a resposta como documento assinado no blob da empresa alvo e envia a própria URL externa ao WhatsApp do funcionário. `doc\_refused/doc\_deleted/doc\_expired` invalidam a unidade e toda a sessão.

## Evidência

### IDR-02 – PeopleManagement: webhook ZapSign sem assinatura, sem escopo de tenant, com download de URL arbitrária (SSRF) e reenvio ao funcionário  
 `server/Services/PeopleManagement/PeopleManagement.Services/Services/SignDocumentService.cs:117-118, 139-142, 155, 180-186 (+ DocumentController.cs:164-166; FileDownloadService.cs:22; ZapSignWebHookManagementService.cs:43-55,192-231)`

```csharp

```
// DocumentController.cs:164-166 – rota absoluta, sem {company}
[HttpPost("/api/v1/[controller]/webhook")]
[ProtectedResource("document", "webhook")]
public async Task<...> ReceiveWebhook([FromBody] JsonNode request, [FromHeader(Name = "x-requestid")] Guid requestId)
```

```
// SignDocumentService.cs:117 – lookup global por DocumentUnitId, sem CompanyId
var document = await _documentRepository.FirstOrDefaultAsync(x => x.DocumentsUnits.Any(x => x.Id == webhookEvent.DocumentUnitId), ...)
// :139-141 – baixa a URL do corpo e grava como 'documento assinado'
var primaryFile = await _fileDownloadService.DownloadFileFromUrlAsync(webhookEvent.Url, cancellationToken);
var primaryFileName = document.InsertUnitWithoutRequireValidation(webhookEvent.DocumentUnitId, Guid.NewGuid().ToString(), primaryFile.FileExtension);
```

```
// :180-186 – a mesma URL vai por WhatsApp ao funcionário
_whatAppQueueService.EnqueueMediaMessage(phoneNumber: ..., media: url, fileName: fileName);
// FileDownloadService.cs:22
var response = await _httpClient.GetAsync(url, cancellationTokens);
...
```

Não há verificação de HMAC/assinatura do provedor; a única barreira é o Keycloak conceder document#webhook ao token apresentado – e esse token é o client_credentials do client `zapsign` que o próprio sistema grava no cabeçalho do webhook cadastrado na ZapSign (ZapSignWebHookManagementService.cs:43-55), visível a quem acessa o painel ZapSign (ou a quem tiver o secret do client – ver SEC-01). Com ele: POST /api/v1/Document/webhook {event_type: doc_signed, external_id: <DocumentUnitId de qualquer tenant>, signed_file: http://169.254.169.254/... ou http://servico-interno/} faz o servidor baixar a URL (SSRF), gravar a resposta como documento assinado no blob da empresa alvo e enviar a URL ao celular do funcionário. doc_refused/doc_deleted/doc_expired invalida a unidade e toda a sessão de assinatura.

Impacto

Forja de documento assinado em qualquer tenant, invalidação de sessões de assinatura de qualquer tenant, SSRF a partir do servidor (metadados de nuvem, serviços internos) e phishing do funcionário com URL controlada pelo atacante. Precisa de um token com `document#webhook` e do GUID da DocumentUnit.

Sugestão de correção

1. Validar a autenticidade do evento: usar o segredo/assinatura de webhook da ZapSign (ou, no mínimo, um header secreto exclusivo por ambiente, comparado em tempo constante) além do Bearer.
1. Restringir `signed\_file`/`extra\_docs[].signed\_file` a `https://` e a hosts da ZapSign (allowlist configurável), exigir `Content-Type: application/pdf` e limitar tamanho; recusar IPs privados/link-local após resolução de DNS.
1. Resolver a DocumentUnit junto com a CompanyId/EmployeeId persistidos na sessão de assinatura (o `external\_id` deve carregar ou ser cruzado com a company) e recusar se não bater.
1. Enviar ao funcionário o arquivo já armazenado no blob (ou um link assinado da própria plataforma), nunca a URL externa recebida.
1. Gerar `x-requestid` a partir do id do evento ZapSign (ou usar o par evento/unidade) para que a idempotência funcione sem o efeito `Guid.Empty`.
1. Cobrir com teste de integração: evento sem assinatura válida → 401/403; `signed\_file` fora da allowlist → 400; DocumentUnit de outra company → 404.

Critérios de aceite

- [] Evento sem a assinatura/segredo da ZapSign é recusado antes de qualquer acesso ao banco.
- [] `signed\_file` apontando para `http://127.0.0.1`, `http://169.254.169.254` ou domínio fora da allowlist é recusado sem abrir conexão.
- [] DocumentUnit pertencente a outra company que não a da sessão retorna 404 e não altera estado.
- [] A mensagem WhatsApp enviada após assinatura referencia só o blob/URL da plataforma.
- [] Dois eventos legítimos consecutivos sem `x-requestid` são ambos processados (idempotência por id do evento).

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] CompanyController permite ler e editar qualquer empresa por Id (fora do guard de tenant)

****Labels sugeridas:**** `security`, `severity:high`, `people-management`, `idor`

Problema

`CompanyController` está em `api/v1/company` (sem `{company}` na rota), então `RouteAccessRequirement` não tem o que comparar e concede. As queries (`GetCompanySimplefiedAsync`, `GetCompaniesSimplefiedsAsync`, `GetCompanyAsync`) filtram só por `Id`, e `EditCompanyCommandHandler` carrega `request.Id` vindo do corpo. Como todo usuário precisa de `company:view` para ver a própria empresa, qualquer usuário lê CNPJ, e-mail, telefone e endereço de qualquer empresa (`GET /api/v1/company/complete?id=...`, em lote por `/list?id=a&id=b`) e, com `company:edit`, altera razão

social/CNPJ/contato de qualquer empresa.

Evidência

IDR-01 – PeopleManagement: CompanyController sem {company} na rota – lê e edita qualquer empresa por Id livre
 `server/Services/PeopleManagement/PeopleManagement.API/Controllers/CompanyController.cs:10, 40-42, 65-87 (+ CompanyQueries.cs:15,31,48; EditCompanyCommandHandler.cs:17-18)`

```
```csharp
[Route("api/v1/{controller}")] // sem {company}: RouteAccessRequirement concede em silêncio
public class CompanyController(...) : BaseController(logger)
{
 [HttpPut]
 [ProtectedResource("company", "edit")]
 public async Task<ActionResult<EditCompanyResponse>> EditCompany([FromBody] EditCompanyCommand request, ...)
 {
 ...
 }
 [HttpGet("complete")]
 [ProtectedResource("company", "view")]
 public async Task<ActionResult<CompanyDto>> GetCompaniesComplete([FromQuery] Guid id)
 // CompanyQueries.cs:48
 var query = _context.Companies.AsNoTracking().Where(x => x.Id == id);
 // EditCompanyCommandHandler.cs:17
 var company = await _companyRepository.FirstOrDefaultAsync(request.Id, ...)
 ...
}
```

O guard anti-IDOR só atua quando a rota tem o parâmetro `company`; aqui não há. Quem tiver company:view (todo usuário precisa dele para ver a própria empresa) chama GET /api/v1/company/complete?id=<GUID de outra empresa> e recebe CNPJ, e-mail, telefone e endereço completo; GET /list?id=a&id=b enumera em lote. Com company:edit, PUT /api/v1/company com Id alheio no corpo altera razão social, CNPJ e contato de qualquer empresa. GUIDs de empresa circulam no claim `companies` e em toda URL do produto. O próprio cliente Flutter depende deste comportamento (company\_api\_service.dart:28 passa os IDs do claim e o servidor devolve qualquer id pedido).

## ## Impacto

Leitura e escrita cross-tenant de dados cadastrais de empresa com privilégio comum. Os GUIDs de empresa circulam no claim `companies` e em todas as URLs do produto.

## ## Sugestão de correção

1. Mover as rotas para `api/v1/{company}/company` (o cliente já conhece o `companyId`) e filtrar `x.Id == company` nas queries e no handler de edição, ignorando o Id do corpo.
1. Para a listagem usada pelo seletor de empresas, filtrar no servidor pelos IDs do claim `companies` do token (`User.FindAll("companies")`) em vez de aceitar `id[]` livre – ou criar `GET /api/v1/me/companies` no molde do `/me/tenants` do TenantManagement.
1. Adicionar teste de integração: usuário com claim `companies=[A]` recebe 403/404 ao consultar/editar a empresa B.
1. Ampliar o teste de erosão (como o `EndpointProtectionTests` do BillPayment) para o PeopleManagement: toda rota `api/v1` de tenant declara `{company}`.

## ## Critérios de aceite

- [ ] `GET /api/v1/company/complete?id=<B>` com token de A retorna 403 ou 404.
- [ ] `PUT` de edição de empresa com Id de B no corpo e token de A não altera B.
- [ ] Seletor de empresas do Flutter continua funcionando via endpoint filtrado no servidor.
- [ ] Teste automatizado que falha se uma rota `api/v1` de tenant nascer sem `{company}`.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---



# [Segurança] PeopleManagement: IDs relacionados sem validação de tenant e joins sem CompanyId

**\*\*Labels sugeridas:\*\*** `security`, `severity:medium`, `people-management`, `tenant-isolation`

### ## Problema

Handlers aceitam do corpo `RoleId`, `WorkPlaceId`, `DepartmentId`, `PositionId`, `DocumentGroupId` e `DocumentsTemplatesIds` sem verificar que pertencem à `CompanyId` da rota (ex.: `AlterRoleEmployeeCommandHandler` faz `employee.RoleId = request.RoleId`). Repositórios e queries seguem esses IDs sem filtrar a entidade destino pela company (`RoleRepository.GetRoleFromEmployeeId` filtra o empregado, não a role; subqueries em `DepartmentQueries`, `DocumentGroupQueries`, `RequireDocumentsQueries`, `DocumentTemplateQueries`, `EmployeeQueries`, `BatchDownloadQueries`). `RequireDocumentsQueries.GetAllWithDocumentList` ainda busca o funcionário sem company.

### ## Evidência

### ISO-01 – PeopleManagement: IDs relacionados aceitos do corpo sem validar tenant, e joins que os seguem sem CompanyId

`server/Services/PeopleManagement/PeopleManagement.Infra/Repository/RoleRepository.cs:13-16 (+ AlterRoleEmployeeCommandHandler.cs:24-27, DepartmentQueries.cs:25-34, DocumentGroupQueries.cs:85-88, RequireDocumentsQueries.cs:106-108, DocumentTemplateQueries.cs:95-96, EmployeeQueries.cs:26, BatchDownloadQueries.cs:20-26)`

```
```csharp
```

```
// RoleRepository.cs:13-16 – filtra o EMPREGADO pela company, não a ROLE
```

```
var query = from e in _context.Employees
             join r in _context.Roles on e.RoleId equals r.Id
             where e.Id == employeeId && e.CompanyId == companyId
             select r;
```

```
// AlterRoleEmployeeCommandHandler.cs:24-27 – RoleId do corpo vai direto para o agregado
```

```
var employee = await _employeeRepository.FirstOrDefaultAsync(x => x.Id == request.EmployeeId &&
x.CompanyId == request.CompanyId, ...)
    ?? throw new DomainException(...);
employee.RoleId = request.RoleId;
```
```

Um usuário com employee:edit na empresa A informa o RoleId (GUID) de uma Role da empresa B em PUT /api/v1/{A}/employee/Role. Nada valida que a Role pertence a A. Em seguida, GET employee/list, batch-download/employees e principalmente a geração de documentos (RecoverRoleInfoToDocumentTemplateService.cs:17-29) seguem o join sem filtrar r.CompanyId e trazem nome, CBO e remuneração (salário base) da Role alheia para o JSON/PDF do documento. O mesmo padrão vale para WorkPlaceId, Position.DepartmentId, Role.PositionId, DocumentTemplate.DocumentGroupId e RequireDocuments.DocumentsTemplatesIds (subqueries em DepartmentQueries.cs:25-34, DocumentGroupQueries.cs:87, RequireDocumentsQueries.cs:106-108 sem CompanyId).

### ISO-02 – PeopleManagement: GetAllWithDocumentList busca o funcionário sem CompanyId

`server/Services/PeopleManagement/PeopleManagement.Application/Queries/RequireDocuments/RequireDocumentsQueries.cs:27-30`

```
```csharp
```

```
var employeeQuery =
    from em in context.Employees.AsNoTracking()
    where em.Id == employeeId
    select em.GetAllPossibleAssociationIds();
```
```

GET /api/v1/{company}/requireddocuments/withdocuments/{employeeId} resolve as associações (cargo, função, local) de um funcionário de QUALQUER empresa. As linhas 35 e 44 limitam o retorno aos RequireDocuments/Documents da company da rota, então o que vaza é apenas 'quais dos meus requisitos batem com as associações de um funcionário estrangeiro' – inferência, não dado direto.

### ## Impacto

Um usuário com `employee:edit` que conheça o GUID de uma Role de outra empresa passa a ver nome, CBO e remuneração dela em listagens e – via `RecoverRoleInfoToDocumentTemplateService` – dentro dos

documentos gerados. O mesmo vale para Position/Department/Workplace/DocumentGroup/DocumentTemplate. Exige conhecer GUIDs alheios.

#### ## Sugestão de correção

1. Em todo handler que atribui um ID relacionado, validar com ``ExistsAsync(id, companyId)`` no repositório correspondente antes de atribuir; lançar ``DomainErrors.ObjectNotFound`` caso contrário.
1. Nos repositórios/queries listados, acrescentar ``&& r.CompanyId == companyId`` (ou equivalente) na entidade destino do join/subquery.
1. Corrigir ``RequireDocumentsQueries.GetAllWithDocumentList`` para ``em.Id == employeeId && em.CompanyId == companyId``.
1. Avaliar ``HasQueryFilter`` por tenant nos DbContexts (com ``IgnoreQueryFilters`` só nos workers) como rede de segurança para regressões.
1. Testes de integração: atribuir RoleId de outra company retorna 404; listagens não retornam nomes de entidades de outra company.

#### ## Critérios de aceite

- [ ] ``PUT employee/Role`` com RoleId de outra company retorna 404 e não altera o funcionário.
- [ ] Documento gerado para funcionário cuja Role foi (indevidamente) apontada para outra company não contém dados dessa Role.
- [ ] Todas as subqueries/joins citados filtram CompanyId (revisão de código + teste).
- [ ] Teste de integração cross-company para pelo menos Role, Workplace e DocumentTemplate.

### --- FIM ISSUE 5 ---

### --- ISSUE 6 ---

# [Segurança] Idempotência por x-requestid é global entre tenants e o header ausente vira Guid.Empty

**\*\*Labels sugeridas:\*\*** ``security``, ``severity:medium``, ``people-management``, ``bill-payment``, ``tenant-management``

#### ## Problema

No PeopleManagement o header ``x-requestid`` é bindado como ``Guid`` (value type) sem ``[BindRequired]``; ausente vira ``Guid.Empty``. ``IdentifiedCommandHandler`` consulta a tabela ``client_requests`` (PK só ``Id``, sem CompanyId/usuário) e, se o Id já existe, devolve ``default`` sem executar e sem erro. Assim, após a primeira requisição sem header de qualquer tenant, toda requisição sem header de todos os tenants (inclusive o webhook ZapSign, que só envia Authorization) vira no-op silencioso. Quem conhecer um requestId de outro tenant pode registrá-lo antes e anular o comando. No BillPayment e TenantManagement o ledger também é global (sem TenantId), embora o header vazio gere Guid novo.

#### ## Evidência

### ISO-03 – PeopleManagement: ledger de idempotência global sem tenant, e x-requestid ausente vira Guid.Empty  
``server/Services/PeopleManagement/PeopleManagement.Application/Commands/Identified/IdentifiedCommandHandler.cs:19-26 (+ EmployeeController.cs:42, RequestManager.cs:11-17, ClientRequestMap.cs:12-14)``

```
```csharp
```

```
// EmployeeController.cs:42 – header opcional; Guid é value type → ausente = Guid.Empty
public async Task<ActionResult<CreateEmployeeResponse>> Create([FromRoute] Guid company, [FromBody]
CreateEmployeeModel request, [FromHeader(Name = "x-requestid")] Guid requestId)
```

```
// IdentifiedCommandHandler.cs:19-26
```

```
var alreadyExists = await _requestManager.ExistsAsync(request.Id);
if (alreadyExists)
{
    _logger.LogInformation("Request {Id} already exists", request.Id);
    return CreateResultForDuplicateRequest(); // default! – não executa e não erra
}
await _requestManager.CreateRequestForCommandAsync<T>(request.Id);
```
```

A tabela `client_requests` tem só `Id/Name/Time`, sem `CompanyId` nem usuário. (1) A primeira requisição sem header, de qualquer tenant, grava `Guid.Empty`; a partir daí TODA requisição sem header de TODOS os tenants (inclusive o webhook ZapSign, que só envia `Authorization`) retorna default sem executar – negação de serviço silenciosa entre tenants. (2) Quem conhecer/adivinhar o `x-requestid` de outro tenant pode registrá-lo antes e anular o comando dele. `BaseController.ValidityRequestId` (`BaseController.cs:51-55`) existe, nunca é chamado e está com a condição invertida; só `CompanyController` valida o header.

### ISO-04 – `BillPayment` e `TenantManagement`: ledger de idempotência também é global (chave escolhida pelo cliente)

``server/Services/BillPayment/BillPayment.Infra/Idempotency/RequestManager.cs:15-16 (+ TenantManagement.Infra/Persistence/ClientRequest.cs:92, RequestManager.cs:77-84)``

```
```csharp
```

```
public Task<bool> ExistAsync(Guid id, CancellationToken ct)
    => db.ClientRequests.AnyAsync(r => r.Id == id, ct); // PK só em Id, sem TenantId
```
```

Mesmo desenho de ISO-03, mas aqui o controller gera um `Guid` novo quando o header vem vazio (`EnsureRequestId`), então não há o efeito `Guid.Empty`. Resta a colisão deliberada: um `Guid` já gasto por qualquer tenant faz o comando de outro tenant devolver resposta neutra sem executar. Exige adivinhar um `Guid` escolhido pelo cliente – não explorável na prática, mas cross-tenant por construção.

## Impacto

Negação de serviço silenciosa entre tenants (comandos aparentam sucesso com resposta vazia) e anulação direcionada de comandos alheios. Afeta também o processamento de webhooks de assinatura.

## Sugestão de correção

1. `PeopleManagement`: recusar ``Guid.Empty`` (400) ou gerar um `Guid` novo quando ausente, como ``EnsureRequestId`` do `BillPayment`; remover/corrigir ``BaseController.ValidityRequestId`` (condição invertida e nunca chamada).
1. Escapar o ledger por tenant: chave composta (``CompanyId`/`TenantId``, ``RequestId``) nos três BCs, ou incluir o ``sub`` do usuário.
1. Devolver 409/``Conflict`` (ou o resultado original) em duplicata, em vez de ``default`` silencioso, para o cliente perceber.
1. Para o webhook ZapSign, derivar o `requestId` do id do evento do provedor.

## Critérios de aceite

- [ ] Duas requisições sem ``x-requestid`` para o mesmo endpoint são ambas executadas (ou a segunda recebe 400 explícito).
- [ ] Mesmo ``x-requestid`` enviado por tenants diferentes executa nos dois.
- [ ] Duplicata real dentro do mesmo tenant retorna resposta idempotente não-vazia ou 409.
- [ ] Migração cria a chave composta sem quebrar registros existentes.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

# [Segurança] Convite de tenant pode ser sequestrado por troca de e-mail no Keycloak

**Labels sugeridas:** ``security``, ``severity:medium``, ``tenant-management``, ``keycloak``

## Problema

``KeycloakTenantAccessProvisioner`` procura o usuário por e-mail (``users?email=...&exact=true``) e, se existir, grava os atributos ``tenants`/`bp_tenants`/`pm_tenants`` nele sem checar ``emailVerified``. O realm exportado tem ``verifyEmail=false``, ``duplicateEmailsAllowed=false`` e permite ao próprio usuário editar ``email`` no Account Console. Uma conta qualquer do realm troca o e-mail para o do futuro convidado; quando o operador concede acesso ao tenant, o claim é gravado na conta do atacante, nenhum convite é enviado (só há e-mail quando o usuário é criado) e a vítima nunca pode ser cadastrada com aquele e-mail. ``GET /me/tenants`` também é chaveado pelo claim ``email``.

```
Evidência
ISO-06 – TenantManagement: claim `tenants` é gravado em qualquer conta que tenha o e-mail –
convite sequestrável por troca de e-mail self-service
`server/Services/TenantManagement/TenantManagement.Infra/Identity/KeycloakTenantAccessProvisioner.cs:6
6-82 (+ 184-195, 135-153; utils/KeyCloakConfig/RufinoRealm/realm-export-2026-08-18.json:35,3109)`
```

```
```csharp
var user = await FindUserByEmailAsync(email, cancellationToken); // users?email=...&exact=true →
users[0]
var created = false;
if (user is null)
{
    await CreateUserAsync(email, tenant, products, cancellationToken);
    ...
}
else
{
    await WriteAccessAsync(user, tenant, products, cancellationToken); // grava
tenants/bp_tenants/pm_tenants sem olhar EmailVerified
}
```
```

O realm exportado tem `verifyEmail=false` (L35), `duplicateEmailsAllowed=false` e o User Profile permite ao próprio usuário editar ``email`` (L3109: `permissions.edit = [admin, user]`) com `account-console` habilitado. Qualquer conta existente no realm troca o próprio e-mail para `fulano@empresa-b.com` ANTES do convite. Quando o operador chama `POST /tenants/{B}/members {email}`, o adapter encontra a conta do atacante e grava `tenants=[B]` nela; nenhum convite é enviado (só quando `created`) e a vítima nunca poderá ser criada com aquele e-mail. `GET /me/tenants` (`MeController.cs:27-32`) também é chaveado pelo `claim email` do token.

#### ## Impacto

Acesso a um tenant por quem não foi convidado; bloqueio permanente do e-mail legítimo. Exige conta prévia no realm e conhecer o e-mail alvo antes do convite.

#### ## Sugestão de correção

1. No provisionador, só gravar claims se ``EmailVerified == true``; caso contrário, criar um registro de convite pendente e disparar ``VERIFY_EMAIL``, gravando o claim apenas após a verificação (ou resolver por ``sub`` informado pelo próprio usuário ao aceitar o convite).
1. No realm: remover ``user`` de ``permissions.edit`` do atributo ``email`` no User Profile (ou ligar ``verifyEmail=true`` para que a troca exija verificação).
1. Em ``/me/tenants`` e em ``TenantQueries.ListForMemberAsync``, chavear por ``IdentityUserId`` (`sub`) além do e-mail.
1. Atualizar ``tenant-management-authz-config.json`/realm` import versionado com a mudança de permissão.

#### ## Critérios de aceite

- [ ] Conceder acesso a e-mail que pertence a uma conta com ``emailVerified=false`` não grava o claim (fica pendente) e registra o motivo.
- [ ] Usuário comum não consegue alterar o próprio e-mail no Account Console (ou a alteração exige verificação).
- [ ] ``/me/tenants`` não lista tenants para uma conta cujo e-mail coincide mas cujo ``sub`` não é membro.
- [ ] Teste de integração cobrindo o cenário de conta pré-existente não verificada.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

# [Segurança] Hangfire Dashboard fora do Keycloak com senha vazia por padrão, e APIs sem validação de segredos no startup

**\*\*Labels sugeridas:\*\*** ``security``, ``severity:medium``, ``people-management``, ``ops``

**## Problema**  
 `/hangfire` é protegido apenas por Basic Auth com `Login=admin` fixo, `PasswordClear` lido de `HangfireDashboard:Password` (vazio no `appsettings.json`), `RequireSsl=false`, montado antes de `UseHttpsRedirection` e fora do pipeline do Keycloak. Nenhuma das três APIs valida configuração ao iniciar (sem `ValidateOnStart`/`IValidateOptions`): `Keycloak:Credentials:secret`, connection strings, chaves de S3/ZapSign/WhatsApp e a senha do Hangfire são aceitos vazios. Existe ainda um `HangFireAuthorizationFilter` que retorna `true` incondicionalmente (não referenciado).

#### ## Evidência

### PRM-01 – Hangfire Dashboard fora do Keycloak, com Basic Auth de senha vazia por padrão e `RequireSsl=false`  
 `server/Services/PeopleManagement/PeopleManagement.API/Program.cs:165-182 (+ appsettings.json:51-54; Filters/HangFireAuthorizationFilter.cs:8-12)`

```
```csharp
app.UseHangfireDashboard("/hangfire", new DashboardOptions
{
    Authorization = new[] { new BasicAuthAuthorizationFilter(new BasicAuthAuthorizationFilterOptions
    {
        RequireSsl = false,
        SslRedirect = false,
        LoginCaseSensitive = true,
        Users = new [] { new BasicAuthAuthorizationUser {
            Login = builder.Configuration["HangfireDashboard:Login"],           // "admin" no
appsettings
            PasswordClear = builder.Configuration["HangfireDashboard:Password"] // "" no appsettings
        } }
    }) }
});
// appsettings.json:51-54
"HangfireDashboard": { "Login": "admin", "Password": "" }
```
```

O dashboard permite disparar/apagar jobs (geração recorrente de documentos, refresh de webhook, lembretes WhatsApp) de todos os tenants e é montado antes de `UseHttpsRedirection` e fora do pipeline Keycloak. Login fixo `admin`; a senha depende exclusivamente da env var `HangfireDashboard\_\_Password` – sem validação de startup. O `HangFireAuthorizationFilter` que retorna true incondicional existe no projeto (código morto) e é uma armadilha para quem trocar o filtro.

### SEC-06 – Nenhuma API rejeita segredo vazio ao iniciar (sem `ValidateOnStart`/`IValidateOptions`)  
 `server/Services/PeopleManagement/PeopleManagement.API/Authentication/AuthenticationExtesion.cs:22 (+ BillPayment/TenantManagement AuthenticationExtesion.cs:23; KeycloakAuthenticationOptions.cs Secret = string.Empty; Program.cs PM:176-177)`

```
```csharp
configuration.GetSection(KeycloakAuthenticationOptions.Section).Bind(keycloakAuthenticationOptions);
// só Bind; Secret vazio passa
// grep ValidateOnStart|ValidateDataAnnotations|IValidateOptions → 0 ocorrências nas 3 APIs
```
```

Keycloak:Credentials:secret, HangfireDashboard:Password, ConnectionStrings, S3, ZapSign e WhatsApp são aceitos vazios; a falha só aparece na primeira permissão avaliada ou no primeiro envio. O BillPayment mitiga com o padrão 'desligado falha alto' (UnconfiguredSecretVault/Storage), mas isso não cobre o secret do Keycloak. Um deploy com env var faltando sobe 'saudável' com os defaults de SEC-02/PRM-01.

#### ## Impacto

O dashboard dispara/apaga jobs de todos os tenants (geração de documentos, webhooks, WhatsApp). Um deploy sem `HangfireDashboard\_\_Password` sobe 'saudável' com credencial vazia (comportamento exato da lib com senha vazia não verificado). Credenciais Basic trafegam em HTTP se o TLS terminar antes.

#### ## Sugestão de correção

1. Proteger `/hangfire` com a mesma autorização do Keycloak (`IDashboardAuthorizationFilter` que valida o JWT e um escopo `hangfire:view`) ou, no mínimo, `RequireSsl=true` + IP allowlist; mapear o dashboard depois de `UseAuthentication/UseAuthorization`.

1. Apagar `Filters/HangFireAuthorizationFilter.cs``.
1. Adicionar `builder.Services.AddOptions<KeycloakAuthenticationOptions>().Bind(...).Validate(o => !string.IsNullOrEmpty(o.Credentials.Secret)).ValidateOnStart()`` nas três APIs, e o equivalente para `HangfireDashboard:Password`` (quando `!IsDevelopment()`), connection strings e chaves de integração.
1. Falhar o startup com mensagem clara indicando a variável ausente.

#### ## Critérios de aceite

- [ ] API não sobe em Production sem `Keycloak__Credentials__secret`` e sem `HangfireDashboard__Password`` (erro explícito no log).
- [ ] `GET /hangfire`` sem credencial válida retorna 401; com JWT sem o escopo retorna 403.
- [ ] `HangFireAuthorizationFilter`` removido do projeto.
- [ ] Teste de integração da fábrica de testes continua passando com as opções válidas.

### --- FIM ISSUE 8 ---

### --- ISSUE 9 ---

# [Segurança] BillPayment: importação JSON aceita StorageKey/SourceId/ContentHash do cliente

**\*\*Labels sugeridas:\*\*** ``security``, ``severity:medium``, ``bill-payment``

#### ## Problema

`ImportBillModel`` (JSON) repassa `SourceId``, `ExternalMessageId``, `ContentHash`` e `StorageKey`` do corpo ao `ImportBillCommand``; o handler não valida que `SourceId`` pertence ao tenant e usa `StorageKey`` como chave do artefato quando não há upload. A variante multipart já zera `ContentHash/StorageKey`` por reconhecer o risco. A única barreira cross-tenant é a checagem de prefixo em `S3AttachmentStorage.EnsureBelongsToTenant``, que sai como 500 (confirma a existência do objeto). Intra-tenant, uma `StorageKey`` de item `ForeignPayer`` – que o endpoint de quarentena recusa servir – pode ser baixada por `GET /bills/{id}/artifact``. `HintSourceId`` em `POST /expectations`` também não é validado (sem vazamento).

#### ## Evidência

### IDR-03 – BillPayment: POST /bills/import (JSON) aceita StorageKey, ContentHash, SourceId e ExternalMessageId do corpo sem validar pertencimento  
`server/Services/BillPayment/BillPayment.Application/Models/Bills/BillModels.cs:16-38 (+ ImportBillCommand.cs:84-98; BillQueries.cs:170-171; S3AttachmentStorage.cs:145-149)``

````csharp`

```
public sealed record ImportBillModel(
    string? DigitableLine, string? PixPayload, string SourceKind,
    [property: JsonRequired] DateTime ReceivedAt,
    Guid? SourceId, string? SenderAddress, string? ExternalMessageId,
    string? ContentHash, string? StorageKey) // ← vêm do cliente
// ImportBillCommand.cs:84-85
var storageKey = document is null ? request.StorageKey : await storage.StoreAsync(...);
// S3AttachmentStorage.cs:145-149 – única barreira cross-tenant, e vira 500
if (!storageKey.StartsWith(Prefix(tenantId), StringComparison.Ordinal))
    throw new UnauthorizedAccessException("A chave de armazenamento não pertence a este tenant.");
...
```

A variante multipart (`ImportBillFormModel`, `BillModels.cs:88-92`) já zera `ContentHash/StorageKey` por reconhecer o risco; a JSON não. Um usuário com `bill:import` cria uma `Bill` cuja origem aponta para um `SourceId` alheio (gravado como 'prova de origem') e para qualquer `StorageKey`. Cross-tenant é barrado apenas pela checagem de prefixo (que sai como 500, confirmando a existência do objeto); intra-tenant nada barra: quem conhecer a `StorageKey` de um item `ForeignPayer` (documento de outro pagador lido pela caixa compartilhada, que o endpoint de quarentena recusa servir) o baixa por `GET /bills/{id}/artifact``. `HintSourceId` em `POST /expectations` tem a mesma ausência de validação, sem vazamento (IDR-04).

IDR-04 – BillPayment: HintSourceId do corpo não é validado contra o tenant em `POST /expectations``
`server/Services/BillPayment/BillPayment.Application/Expectations/Commands/RegisterBillExpectationComm`

and.cs:66`

```
```csharp
request.HintSourceId is { } sourceId ? CaptureSourceId.From(sourceId) : null, // sem
captureSources.GetAsync(tenantId, sourceId)
```
```

PayeeId no mesmo handler É validado (L48); HintSourceId não. O único consumidor é ListByHintSourceAsync(tenantId, sourceId) com filtro de tenant, logo um GUID alheio nunca casa – efeito é dado inconsistente no próprio tenant, não IDOR.

Impacto

Evidência de origem forjada (SourceId alheio gravado como prova), contorno do gate `ForeignPayer` do ADR-008 para quem conhecer uma StorageKey, e oráculo de existência de objetos de outro tenant via 500.

Sugestão de correção

1. Fazer `ImportBillModel.ToCommand` passar `ContentHash: null, StorageKey: null`, igual ao multipart.
1. Quando `SourceId` (ou `HintSourceId`) vier informado, validar com `ICaptureSourceRepository.GetAsync(tenantId, sourceId)` e lançar erro de domínio se não existir; ou recusar esses campos fora de `SourceKind = ManualUpload` na API pública.
1. Mapear `UnauthorizedAccessException` do storage para 404 no `DomainExceptionFilter` (ou lançar erro de domínio próprio).
1. Teste de integração: importação JSON com StorageKey de outro tenant → Bill sem artefato; SourceId inexistente → 400.

Critérios de aceite

- [] Bill importada via JSON nunca nasce com `Origin.StorageKey` vindo do cliente.
- [] `SourceId`/`HintSourceId` de outro tenant retornam erro de domínio (400/404), nunca gravam.
- [] `GET /bills/{id}/artifact` nunca responde 500 por chave de outro tenant.
- [] Documentação (CLAUDE.md do BillPayment, seção de importação) atualizada.

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

[Segurança] Defaults de credencial em appsettings/compose/template.env e AccessKey de produção versionado

****Labels sugeridas:**** `security`, `severity:medium`, `secrets`, `ops`

Problema

A senha do Postgres `5up3rS3nh@` está no `appsettings.json` BASE das três APIs e em 9 pontos dos docker-compose (que publicam os bancos em 0.0.0.0); o Keycloak local sobe com `Admin/Admin` em `0.0.0.0:8082` importando o realm completo; MinIO usa `billpayment/billpayment-dev`; o `template.env` da Evolution API traz `AUTHENTICATION\_API\_KEY=senha-aqui` e `CORS\_ORIGIN=\*`, e o `deploy.ps1` injeta o `.env` inteiro como `--env-vars` (texto claro no Container App) em vez de `--secrets`. O `appsettings.json` do PeopleManagement ainda versiona o Access Key ID do Garage de produção e um telefone real.

Evidência

SEC-02 – Senha do Postgres embutida no appsettings.json base das 3 APIs e em todos os docker-compose

`server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:9-10 (+ BillPayment.API/appsettings.json:10; TenantManagement.API/appsettings.json:10; server/docker-compose.yml:68,91,111,132,168,239,259; BillPayment/docker-compose.yml:12,52; TenantManagement/docker-compose.yml:12,26)`

```
```json
```

```
"ConnectionStrings": {
 "Postgresql":
```

```
"Server=peoplemanagement.db;Port=5432;Database=PeopleManagementDb;SearchPath=people_management;UserId=
```

```
admin;Password=5up3rS3nh@",
 "HangfireConnection":
"Server=peoplemanagement.db;Port=5432;Database=hangfiredb;UserId=admin;Password=5up3rS3nh@"
}
// docker-compose.yml:68 POSTGRES_PASSWORD: "5up3rS3nh@" (mesma senha desde 2022, em 9 lugares)
...
```

É default de desenvolvimento, mas está no appsettings BASE (não no Development): qualquer ambiente que não sobrescreva ConnectionStrings\_\_X sobe com esta senha, e nenhuma API valida a configuração ao iniciar. Os composes publicam os bancos em 0.0.0.0 (8042/8102/8112/8081) – na LAN, qualquer máquina entra como admin. A senha também é lida no PDF gerado a partir de appsettings.json via XSS-01.

### SEC-03 – Keycloak local com admin Admin/Admin e MinIO com credenciais fracas, publicados no host `server/docker-compose.yml:230-231, 239, 189-190, 135 (+ BillPayment/docker-compose.yml:15,73-74; portas 8082 e 8104 em :193,244; server/.env:13 aponta 192.168.15.41)`

```
```yaml
KC_BOOTSTRAP_ADMIN_USERNAME: Admin
KC_BOOTSTRAP_ADMIN_PASSWORD: Admin
KC_DB_PASSWORD: 5up3rS3nh@
...
MINIO_ROOT_USER: "billpayment"
MINIO_ROOT_PASSWORD: "billpayment-dev"
Storage__SecretKey: "billpayment-dev"
```
```

O Keycloak local importa o realm `rufino` completo (clients, usuários, SMTP mascarado) e fica em 0.0.0.0:8082 – com Admin/Admin qualquer máquina da LAN vira administrador do realm e pode emitir tokens aceitos pelas APIs locais. O .env do servidor referencia o IP de LAN 192.168.15.41, indicando que o compose é acessado por outras máquinas.

### SEC-04 – Evolution API (Azure): template.env com API key placeholder e CORS \*, e deploy.ps1 injeta segredos como env vars em texto claro  
`azure/container-apps/evolution-api/template.env:8, 24, 38 (+ deploy.ps1:92; main.bicep:29-31; container-app.yml:8)`

```
```bash
AUTHENTICATION_API_KEY=senha-aqui
CORS_ORIGIN=*
DATABASE_CONNECTION_URI=postgresql://user:password@evolution.db:5432/evolution_api
# deploy.ps1:92
$azCommand += '--env-vars'      # lê o .env inteiro e passa como variáveis comuns, não --
secrets/secretref
```
```

A Evolution API é o gateway de WhatsApp da plataforma e tem ingress externo (main.bicep:29-31). Se alguém copiar template.env → .env sem trocar a chave, a API sobe aceitando `senha-aqui` como chave global com CORS aberto. Mesmo com chave forte, o deploy.ps1 a grava como variável de ambiente comum do Container App (visível no portal/az CLI) em vez de secretref – o main.bicep até marca environmentVariables como @secure(), mas o script não usa o bicep.

### SEC-07 – Access Key ID do S3 (Garage) de produção e número de telefone reais versionados  
`server/Services/PeopleManagement/PeopleManagement.API/appsettings.json:13-14, 62`

```
```json
"S3": {
  "ServiceURL": "https://garage.couratechsafety.cloud",
  "AccessKey": "GKcf..e931",
  "SecretKey": "",
  ...
"WhatsApp": { ..., "HealthCheckNumber": "5511968608425" }
```
```

O Access Key ID sozinho não autentica (SecretKey vazia), mas é metade da credencial de produção e revela o endpoint do storage; o telefone é PII de pessoa real. Ambos deveriam viver em configuração de ambiente.



**## Impacto**

Qualquer ambiente sem override sobe com credenciais conhecidas; na LAN de desenvolvimento qualquer máquina vira admin do Postgres/Keycloak/MinIO; a chave da Evolution API pode ir para produção como `senha-aqui` e, mesmo forte, fica legível na configuração do Container App.

**## Sugestão de correção**

1. Mover senhas de connection string para `appsettings.Development.json`/user-secrets e deixar o `appsettings.json` base sem credencial (combinar com a validação de startup da issue do Hangfire).
1. Nos composes, usar `\${POSTGRES\_PASSWORD:?}`, `\${KC\_ADMIN\_PASSWORD:?}`, `\${MINIO\_ROOT\_PASSWORD:?}` lidos do `.env` ignorado; publicar portas em `127.0.0.1` por padrão.
1. Remover `S3.AccessKey` e `WhatsApp.HealthCheckNumber` do `appsettings.json` (env var/secretref).
1. No `deploy.ps1` da Evolution API, separar segredos (`--secrets` + `secretref:`) das variáveis comuns e trocar `CORS\_ORIGIN=\*` por origens explícitas; no `template.env`, deixar `AUTHENTICATION\_API\_KEY=` vazio com comentário obrigatório.
1. Documentar no CLAUDE.md do server a política 'nenhuma credencial no appsettings base'.

**## Critérios de aceite**

- [ ] `grep -rn "5up3rS3nh@|Admin\b|billpayment-dev|senha-aqui" server azure` retorna só ocorrências em comentários/exemplos, nunca em valor efetivo.
- [ ] `docker compose up` falha com mensagem clara se as variáveis de senha não estiverem definidas.
- [ ] Container App da Evolution API mostra `AUTHENTICATION\_API\_KEY` como secretref.
- [ ] `appsettings.json` do PeopleManagement sem AccessKey e sem telefone.

--- FIM ISSUE 10 ---

--- ISSUE 11 ---

# [Segurança] Segredos e dados pessoais em logs (API key do WhatsApp, payload completo dos Commands)

**\*\*Labels sugeridas:\*\*** `security`, `severity:medium`, `people-management`, `tenant-management`, `lgpd`

**## Problema**

`WhatsAppService` grava a API key da Evolution API em `LogInformation` a cada envio (a variável chama-se `apiKeyMasked` mas não mascara), junto com telefone e corpo da mensagem. `BaseController` do PeopleManagement e do TenantManagement logam `{@Command}` inteiro no envio e no resultado – CPF, filiação, endereço, dependentes, e-mails – sem o mecanismo `ISensitiveCommand` que o BillPayment já usa.

**## Evidência**

### SEC-05 – API key da Evolution API (WhatsApp) gravada em log Information a cada mensagem  
 `server/Services/PeopleManagement/PeopleManagement.Infra/Services/WhatsAppService.cs:40-47`

```
```csharp
var apiKeyMasked = _options.ApiKey; // nome diz 'masked', valor é a chave inteira
_logger.LogInformation(
    "Sending WhatsApp message - URL: {FullUrl}, Instance: {Instance}, ApiKey: {ApiKey}, Body: {Body}",
    fullUrl, _options.Instance, apiKeyMasked, bodyJson);
```
```

A chave global da Evolution API (que permite enviar mensagens em nome da instância e ler conversas) vai para o sink de log em texto claro junto com número de telefone e corpo da mensagem (PII). Quem lê logs (Azure Log Analytics, contêiner) obtém a credencial.

### SEC-08 – Payload completo dos Commands (CPF, filiação, endereço, dependentes, e-mails) em log Information  
 `server/Services/PeopleManagement/PeopleManagement.API/Controllers/BaseController.cs:24-49 (+ TenantManagement.API/Controllers/BaseController.cs:59-64; nenhum Command do TM implementa ISensitiveCommand)`

```
```csharp
```

```
_logger.LogInformation(
    "----- Sending command: {CommandName} - {IdProperty}: {CommandId} ({@Command}) - RequestId : {RequestId} -----",
    CommandName, IdProperty, CommandId, Command, RequestId); // {@Command} desestrutura o objeto inteiro
...

```

AlterIdCard, AlterPersonalInfo, CreateDependent, RegisterTenant etc. vão inteiros para o log, duas vezes (envio e resultado). O BillPayment resolveu isso com ISensitiveCommand + omissão do payload no resultado; o PeopleManagement e o TenantManagement não. É exposição de dado pessoal (LGPD) a quem lê logs, não credencial.

Impacto

Quem lê logs (Log Analytics, contêiner, ferramentas de observabilidade) obtém a credencial global do WhatsApp e dados pessoais de funcionários (exposição LGPD).

Sugestão de correção

1. Remover `ApiKey` do log do `WhatsAppService` (ou logar só os 4 últimos caracteres).
1. Portar `ISensitiveCommand` + `Loggable()` do BillPayment para PeopleManagement e TenantManagement e marcar os Commands com PII; não repetir `{@Command}` no log de resultado.
1. Configurar um destructuring policy do Serilog/ILogger que ignore propriedades marcadas como sensíveis, como segunda barreira.

Critérios de aceite

- [] Nenhuma linha de log contém a API key do WhatsApp (teste com sink em memória).
- [] Envio de `AlterIdCardEmployeeCommand` gera log sem CPF/RG (payload omitido).
- [] Documentação do padrão no CLAUDE.md dos dois BCs.

--- FIM ISSUE 11 ---

--- ISSUE 12 ---

[Segurança] Hardening do PeopleManagement: TestController, Swagger em produção, CORS, FallbackPolicy, headers e erros do banco

Labels sugeridas: `security`, `severity:low`, `people-management`, `hardening`

Problema

Itens de defesa em profundidade: (1) `TestController` responde anonimamente em produção e nenhum `Program.cs` define `FallbackPolicy`, então qualquer action sem atributo nasce pública; (2) `UseSwagger/UseSwaggerUI` incondicionais no PeopleManagement; (3) CORS `AllowAnyOrigin` no PeopleManagement e origem dinâmica + credenciais em Development nos outros BCs; (4) `RequireHttpsMetadata=false` por `SslRequired: none`; (5) nginx do Flutter web sem CSP/X-Frame-Options/nosniff/HSTS e pdf.js de CDN sem `integrity`; (6) `ApplicationExceptionHandler` devolve `InnerException.Message` do Postgres ao cliente.

Evidência

PRM-02 – Sem FallbackPolicy nas 3 APIs: action sem atributo é anônima – e o TestController do PeopleManagement está nessa condição em produção
 `server/Services/PeopleManagement/PeopleManagement.API/Controllers/TestController.cs:7-14 (+ ProtectedResourcePolicyProvider.cs:13-16 nos 3 BCs)`

```
```csharp
[Route("[controller]")]
public class TestController(ILogger<TestController> logger) : ControllerBase
{
 [HttpGet]
 public IActionResult GetTest()
 {
 return Ok("It's ok. v5");
 }
}

```

```
// ProtectedResourcePolicyProvider.cs:13-16
public Task<AuthorizationPolicy?> GetFallbackPolicyAsync()
=> Task.FromResult<AuthorizationPolicy?>(null);
...
```

GET /Test responde sem token; /Test/auth e /Test/auth/role usam o recurso fictício `teste` e não têm {company} (guard desligado). Nenhum Program.cs define FallbackPolicy = RequireAuthenticatedUser, então qualquer controller futuro sem atributo nasce público. No BillPayment o teste de erosão só cobre rotas que começam com api/v1 (EndpointProtectionTests.cs:40).

### PRM-03 – PeopleManagement expõe Swagger/OpenAPI incondicionalmente em produção  
`server/Services/PeopleManagement/PeopleManagement.API/Program.cs:189-190`

```
```csharp
app.UseSwagger();
app.UseSwaggerUI();
```
```

Sem checagem de ambiente (BillPayment e TenantManagement só expõem em IsDevelopment()). Publica a superfície completa da API, incluindo o webhook e o TestController, para reconhecimento anônimo.

### PRM-04 – CORS AllowAnyOrigin no PeopleManagement; BillPayment/TenantManagement liberam qualquer origem com credenciais em Development  
`server/Services/PeopleManagement/PeopleManagement.API/Program.cs:103-108 (+ BillPayment.API/Extension/CorsExtensions.cs:16-23; TenantManagement idem)`

```
```csharp
builder.Services.AddCors(options =>
{ options.AddPolicy("CorsPolicy", builder => builder
    .AllowAnyOrigin()
    .AllowAnyMethod()
    .AllowAnyHeader());
});
// CorsExtensions.cs:16-22 (BP/TM)
if (environment.IsDevelopment() && origins.Length == 0)
    policy.SetIsOriginAllowed(_ => true).AllowAnyHeader().AllowAnyMethod().AllowCredentials();
...

```

Com Bearer (sem cookies) o impacto é limitado: uma origem maliciosa só consegue chamar a API com um token que já possua. Ainda assim, AllowAnyOrigin retira a última barreira do navegador para exfiltração via XSS num cliente. Nos outros BCs o ramo de Development combina origem dinâmica com AllowCredentials – inofensivo apenas enquanto o compose ficar na máquina do desenvolvedor (docker-compose.yml:89 define Development).

PRM-07 – RequireHttpsMetadata desligado por configuração (SslRequired: none) nas 3 APIs
`server/Services/PeopleManagement/PeopleManagement.API/Authentication/KeycloakExtesion.cs:13-14 (+ BillPayment KeycloakExtesion.cs:15; TenantManagement KeycloakExtesion.cs:13; appsettings.json "SslRequired": "none")`

```
```csharp
options.RequireHttpsMetadata = string.IsNullOrEmpty(keycloakAuthenticationOptions.SslRequired) ==
false
 && keycloakAuthenticationOptions.SslRequired.Equals("external",
StringComparison.OrdinalIgnoreCase);
// appsettings.json: "SslRequired": "none" → RequireHttpsMetadata = false
...

```

Hoje inócuo porque AuthServerUrl é https://, mas um override Keycloak\_\_AuthServerUrl=http://... (previsto no compose via KEYCLOAK\_URL) passa a ser aceito sem aviso, permitindo obter o JWS por canal não cifrado.

### XSS-03 – Sem cabeçalhos de segurança/CSP no nginx do Flutter web nem nas APIs; pdf.js carregado de CDN sem integrity  
`client/rufino\_v2/nginx/default.conf:9-17 (+ client/rufino\_v2/web/index.html:41-43; grep X-Frame-Options|Content-Security-Policy|UseHsts nas 3 APIs → 0)`

```

```nginx
add_header Cache-Control "no-cache, no-store, must-revalidate" always; # únicos add_header do
arquivo
# index.html:41
<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/2.11.338/pdf.min.js"></script> # sem
integrity/crossorigin
```

Defesa em profundidade: sem CSP/X-Frame-Options/nosniff/HSTS o app web fica sem a segunda barreira
caso surja um vetor de injeção, e o pdf.js de CDN sem SRI executa qualquer código que o CDN sirva. Não
há vetor de XSS no Flutter hoje (ver pontos fortes).

XSS-04 – PeopleManagement devolve a mensagem interna do Postgres ao cliente em erros de banco
`server/Services/PeopleManagement/PeopleManagement.API/Filters/ApplicationExceptionHandler.cs:44 (+ 16-
37)`

```csharp
var message = exception.InnerException?.Message; // Npgsql: nomes de tabela/coluna/constraint → JSON
400 "Errors"
```

Vazamento de informação (esquema do banco) em JSON – não é XSS porque nada é renderizado como HTML,
mas facilita reconhecimento. BillPayment/TenantManagement devolvem só erros de domínio.

Impacto
Reconhecimento facilitado (Swagger, mensagens de banco), ausência de segunda barreira do navegador e
risco de regressão (endpoint futuro sem atributo público).

Sugestão de correção
1. Remover `TestController`; nos três `Program.cs`, `options.FallbackPolicy = new
AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build()` (mantendo `[AllowAnonymous]` no
health).
1. Condicionar Swagger a `IsDevelopment()` no PeopleManagement.
1. CORS: `WithOrigins(config)` no PeopleManagement; em BP/TM manter o ramo permissivo só quando
`IsDevelopment()` E exigir `Cors:AllowedOrigins` fora dele (já é assim, apenas documentar).
1. `RequireHttpsMetadata = !IsDevelopment()` independentemente de `SslRequired`.
1. nginx: `add_header Content-Security-Policy`, `X-Frame-Options DENY`, `X-Content-Type-Options
nosniff`, `Strict-Transport-Security`; `integrity`+`crossorigin` no pdf.js (ou servir localmente).
1. `ApplicationExceptionHandler`: devolver erro de domínio genérico e logar o `InnerException` no
servidor.

Critérios de aceite
- [] `GET /Test` retorna 404; action sem atributo em qualquer BC retorna 401 (teste).
- [] `/swagger` retorna 404 em Production no PeopleManagement.
- [] Resposta 400 de violação de constraint não contém nomes de tabela/coluna.
- [] Cabeçalhos de segurança presentes na resposta do nginx (teste com curl).

```

--- FIM ISSUE 12 ---

--- ISSUE 13 ---

```

[Segurança] Limpeza: código morto perigoso, nomes de recurso divergentes e campos de backfill

Labels sugeridas: `security`, `severity:low`, `tech-debt`

Problema
Itens sem exploração hoje, mas que convidam a regressões: `ResolveDecidingUserId(Guid headerUserId)`
no TenantManagement (fallback de identidade por header, já removido do BillPayment por ADR-007);
`HangFireAuthorizationFilter` allow-all não referenciado; `[ProtectedResource("Archive")]` vs
`"archive"`; gates do Flutter com recurso divergente (`document_group`, `company:edit` para editar
template); `RegisterTenantModel.Id` permite escolher o TenantId (backfill); nenhum DbContext tem
`HasQueryFilter` por tenant; o teste de erosão do BillPayment só cobre rotas `api/v1`.

```

## ## Evidência

### PRM-05 – Nomes de recurso divergentes entre UI e servidor (e dentro do servidor)

```
`server/Services/PeopleManagement/PeopleManagement.API/Controllers/ArchiveController.cs:30 vs 12 (+
client/rufino_v2/.../document_group_with_templates_screen.dart:229;
document_template_list_screen.dart:117)`
```

```
```csharp
```

```
[ProtectedResource("archive", "edit")] // linha 12
[ProtectedResource("Archive", "edit")] // linha 30 – Keycloak distingue maiúsculas
// Flutter: resource: 'document_group' (servidor usa document-group); hasPermission('company','edit')
para editar template (servidor exige document-template:edit)
```

O servidor está correto em todos os casos (o gate real é o [ProtectedResource]); o efeito é funcional: botão sempre oculto ou 403 inesperado. Registrado porque divergência de nome é o que faz alguém 'consertar' afrouxando o servidor.

PRM-06 – Código morto perigoso: fallback de identidade por header no TenantManagement e filtro allow-all do Hangfire

```
`server/Services/TenantManagement/TenantManagement.API/Controllers/BaseController.cs:101-102 (+
PeopleManagement.API/Filters/HangFireAuthorizationFilter.cs:8-12)`
```

```
```csharp
```

```
protected Guid ResolveDecidingUserId(Guid headerUserId)
 => TryGetUserId(out var fromToken) ? fromToken : headerUserId; // nenhum controller chama
// HangFireAuthorizationFilter.cs:8-12
public bool Authorize(DashboardContext context) { return true; } // não referenciado
```

Nenhum dos dois está ligado ao pipeline hoje (grep confirma). O BillPayment removeu o mesmo fallback com justificativa explícita (ADR-007); manter a cópia no TenantManagement convida a reaproveitá-la.

### IDR-05 – TenantManagement: operador pode escolher o TenantId no registro (backfill por desenho)

```
`server/Services/TenantManagement/TenantManagement.Application/Models/Tenants/TenantModels.cs:10-20`
```

```
```csharp
```

```
public sealed record RegisterTenantModel(
    string Kind, string LegalName, string? TradeName, string PrimaryTaxId, string ContactEmail,
    string? ContactPhone, AddressInput Address, string OwnerEmail, IReadOnlyList<string>? Products,
    Guid? Id) // ← vira o TenantId gravado no claim
```

Com tenant#create + tenant-access#edit (papel de plataforma tenant-admin) é possível emitir o claim tenants para um GUID arbitrário – inclusive um CompanyId de outro produto ainda não backfillado. A única defesa é ExistsAsync. É o caminho de backfill previsto; registrado para que o campo seja removido quando o backfill terminar.

ISO-05 – Nenhum DbContext tem HasQueryFilter global: o isolamento depende de cada query

```
`server/Services/BillPayment/BillPayment.Infra/Persistence/BillPaymentDbContext.cs:51-76 (grep
HasQueryFilter vazio nos 3 BCs)`
```

```
```csharp
```

```
// BillPaymentDbContext.cs OnModelCreating: só remove FKs cross-aggregate; nenhum HasQueryFilter(x =>
x.TenantId == ...)
// grep -rn HasQueryFilter server/Services → 0 ocorrências
```

Não é falha por si – o BillPayment provou ponto a ponto que todas as queries filtram – mas significa que não há rede de segurança: uma query nova sem Where(TenantId) vaza dados silenciosamente. Os achados ISO-01/ISO-02 do PeopleManagement são exatamente esse tipo de deslize.

## ## Impacto

Baixo. Risco de alguém 'religar' o fallback por header ou o filtro allow-all, e de divergência de nomes levar a afrouxar o servidor.

**## Sugestão de correção**

1. Remover ``ResolveDecidingUserId(Guid)`` do `TenantManagement` e ``HangFireAuthorizationFilter`` do `PeopleManagement`.
1. Unificar o nome do recurso ``archive`` no ``ArchiveController``; corrigir os gates do Flutter para os nomes do servidor.
1. Remover ``RegisterTenantModel.Id`` quando o backfill terminar (registrar prazo).
1. Avaliar ``HasQueryFilter`` por tenant nos `DbContexts`; ampliar o teste de erosão para todas as rotas mapeadas.

**## Critérios de aceite**

- [ ] Grep por ``headerUserId`` e ``HangFireAuthorizationFilter`` retorna vazio.
- [ ] Botões de editar grupo de documento e de editar template aparecem para quem tem o escopo correto.
- [ ] Teste de erosão falha para rota fora de ``api/v1`` sem ``[ProtectedResource]``/``[AllowAnonymous]``.

--- FIM ISSUE 13 ---